

NOMBRE: ENYA MIRANDA FLORES **FECHA: 25 DE OCTUBRE DEL 2026**

INTRODUCCIÓN:

SQL significa Structured Query Language (Lenguaje de Consulta Estructurado). En palabras muy simples, es el idioma universal que se usa para hablar con las bases de datos. En cualquier organización la información se almacena en estructuras de datos interconectadas. SQL permite buscar, filtrar, ordenar, calcular y unir millones de registros de forma precisa y eficiente.

Este desafío forma parte de una formación integral en tecnología y ciberseguridad.

OBJETIVO DE LA ACTIVIDAD:

Desarrollar competencias sólidas en la extracción, interpretación y análisis de información mediante SQL, comprendiendo el funcionamiento de las bases de datos relacionales para optimizar la toma de decisiones organizacionales.

METODOLOGÍA:

Para el desarrollo práctico de este reto, se utilizó la plataforma interactiva SQL Easy (https://www.sql-easy.com/es/) un entorno basado en la web que permite ejecutar consultas sobre bases de datos relacionales simuladas en tiempo real. La resolución práctica de los 31 ejercicios o lecciones de la plataforma SQL Easy, son documentados de manera individual bajo una ficha técnica de 8 puntos:

1. Número y nombre del ejercicio
2. Objetivo del ejercicio
3. Concepto SQL
4. Consulta ejecutada
5. Resultado
6. Interpretación
7. Aprendizaje
8. Aplicación en la vida real



FICHAS TECNICAS DE LOS 31 EJERCICIOS

Para los ejercicios del 1 al 5 se observó y uso la siguiente tabla disponible:

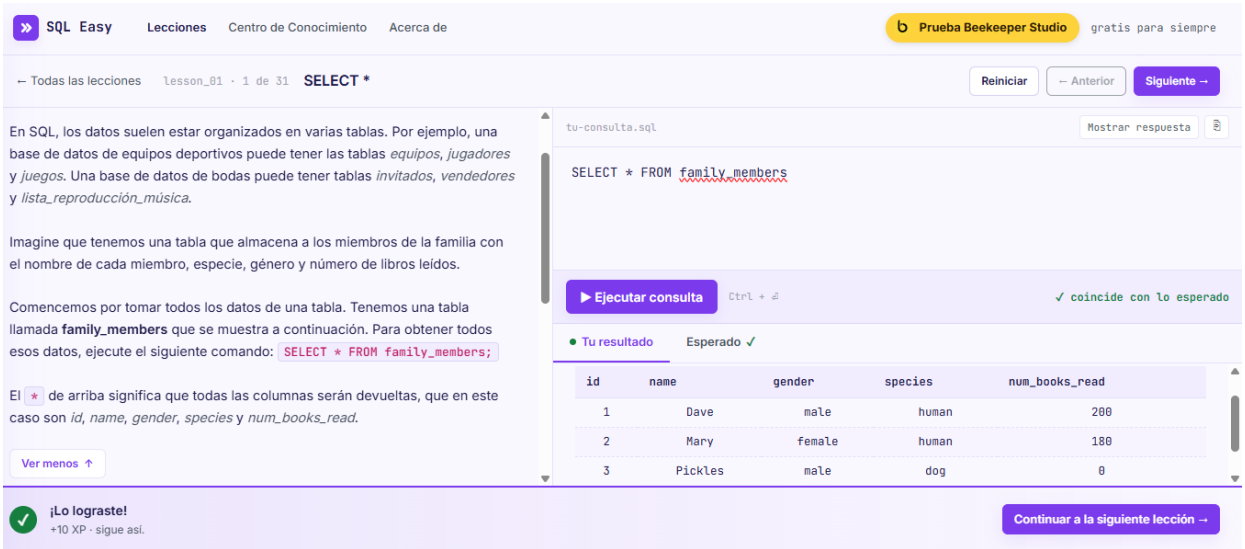
// TABLAS DISPONIBLES — 1 tabla

family_members · 3

id PK int	name char	gender char	species char	num_books_read int
1	Dave	male	human	200
2	Mary	female	human	180
3	Pickles	male	dog	0

Ejercicio 01: SELECT * (Seleccionar todo)

1. **Número y nombre del ejercicio:** Lección 1: SELECT *
2. **Objetivo del ejercicio:** Consultar y mostrar absolutamente todos los datos guardados dentro de la tabla llamada family_members.
3. **Concepto SQL:** Uso del comando básico SELECT combinado con el asterisco (*), el cual funciona para indicar "todo" o "todos los campos".
4. **Consulta ejecutada:** Para obtener todos esos datos, ejecute el siguiente comando: SELECT * FROM family_members SELECT * FROM family_members
5. **Resultado:** Una tabla completa que muestra todas las columnas (id, name, gender, species, num_books_read) y filas con la información de Dave, Mary y Pickles.



6. **Interpretación:** La base de datos responde entregando un panorama general y completo de toda la información disponible en esa tabla sin omitir ningún detalle.

7. **Aprendizaje:** Comprendí la estructura fundamental de una consulta en SQL y cómo el asterisco nos permite visualizar la totalidad de la información de manera inmediata.

8. **Aplicación en el mundo real:** Una empresa u organización, puede abrir una lista general de su inventario para ver todos los registros y campos disponibles antes de empezar a filtrar o buscar algo en específico.

Ejercicio 02: SELECT columnas específicas

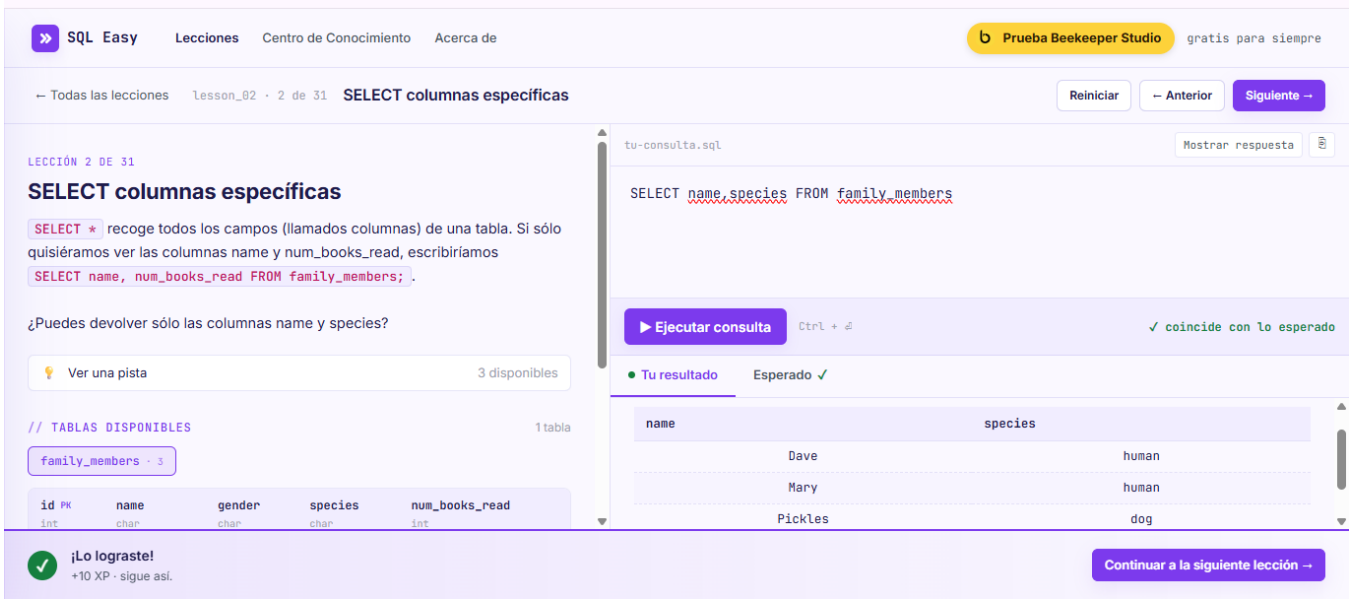
1. **Número y nombre del ejercicio:** Lección 2: SELECT columnas específicas

2. **Objetivo del ejercicio:** Mostrar únicamente la información de los nombres y las especies de los miembros de la familia, sin mostrar los demás datos.

3. **Concepto SQL:** Selección de columnas específicas utilizando los nombres de los campos en lugar de traer toda la tabla completa.

4. **Consulta ejecutada:** ¿Puedes devolver sólo las columnas name y species? SELECT name, species FROM family_members

5. **Resultado:** Una tabla con solo dos columnas (name y species) que enlistan a Dave, Mary y Pickles junto a su respectiva especie.



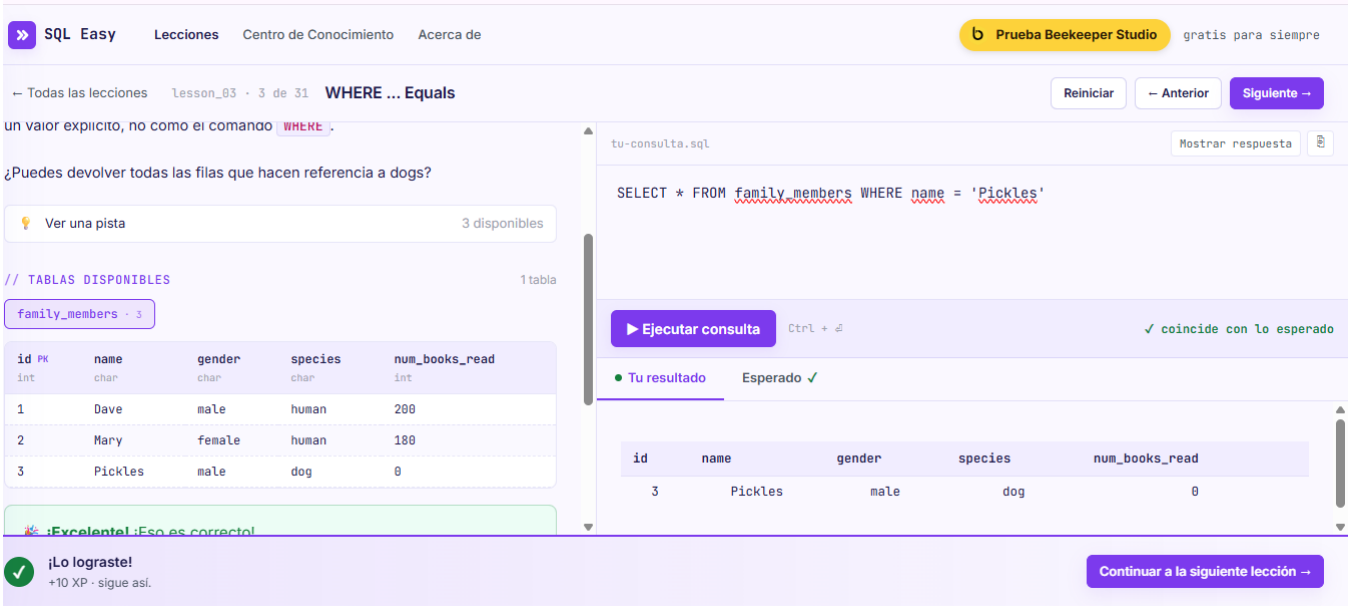
6. **Interpretación:** La base de datos obedece la orden de ignorar columnas que no necesitamos (como el género o los libros leídos) y nos entrega únicamente la información que pedimos.

7. **Aprendizaje:** Comprendí que no siempre es necesario ver todos los datos de una tabla; podemos elegir columnas específicas para tener resultados más ordenados.

8. **Aplicación en el mundo real:** Si en una empresa el área de Recursos Humanos necesita enviar un correo con los nombres y teléfonos de los trabajadores, se puede usar esta consulta para no cargar con datos innecesarios como sus direcciones o contraseñas.

Ejercicio 03: WHERE ... Equals (Igual a)

1. **Número y nombre del ejercicio:** Lección 3: WHERE ... Equals
2. **Objetivo del ejercicio:** Encontrar y mostrar la información completa únicamente del miembro de la familia llamado "Pickles".
3. **Concepto SQL:** Uso de la cláusula WHERE combinada con el signo de igualdad (=) para filtrar registros exactos.
4. **Consulta ejecutada:** ¿Puedes devolver todas las filas que hacen referencia a dogs? SELECT * FROM family_members WHERE name = 'Pickles';
5. **Resultado:** Una sola fila que muestra todos los datos del registro cuyo nombre coincide exactamente con Pickles (ID 3, género male, especie dog, 0 libros leídos).



6. **Interpretación:** El sistema revisa toda la lista de la familia y descarta a los demás, devolviendo únicamente los datos de la persona o mascota que buscamos por su nombre.

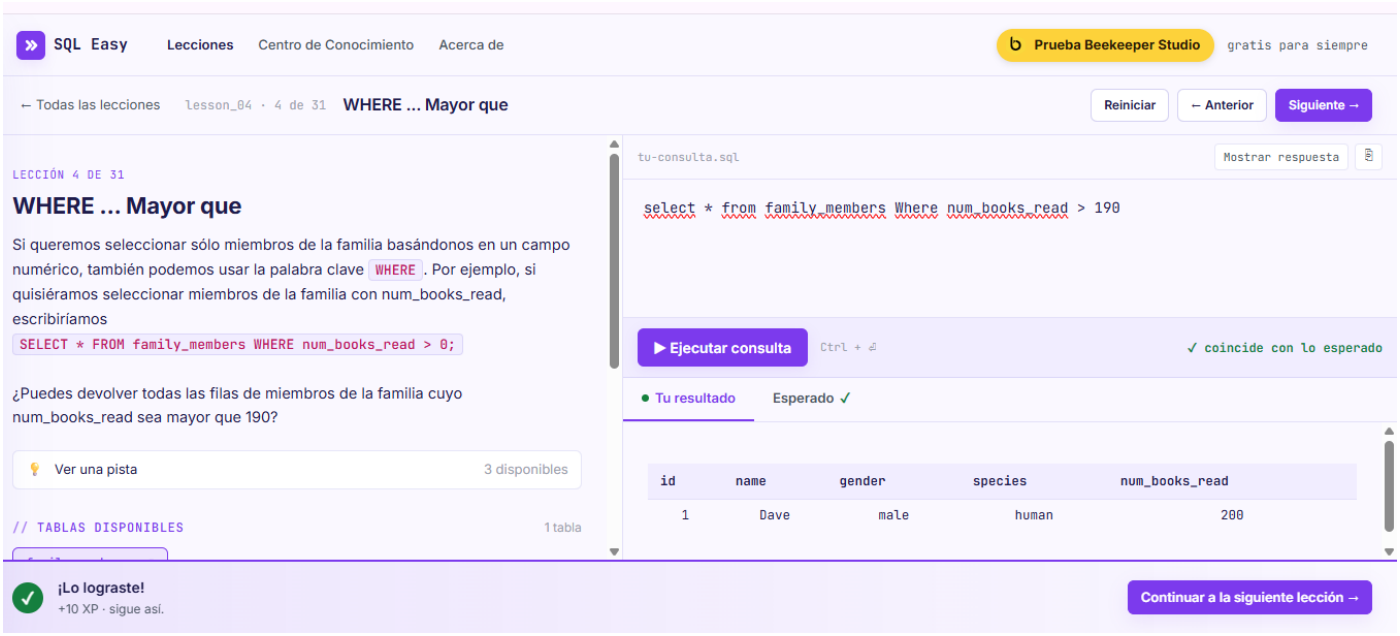
7. **Aprendizaje:** Aprendí a usar el comando WHERE para hacer búsquedas personalizadas y exactas, permitiéndome localizar un registro específico dentro de una gran tabla.

8. **Aplicación en el mundo real:** Cuando buscas un producto por su nombre exacto en una tienda en línea (por ejemplo, "Audífonos Bluetooth") el sistema te muestra solo ese artículo con todas sus características.

Ejercicio 04: WHERE ... Mayor que

1. **Número y nombre del ejercicio:** Lección 4: WHERE ... Mayor que
2. **Objetivo del ejercicio:** Encontrar a los miembros de la familia que hayan leído una cantidad de libros estrictamente mayor a 190.
3. **Concepto SQL:** Uso del operador matemático "mayor que" (>) junto con WHERE para filtrar datos numéricos.
4. **Consulta ejecutada**: ¿Puedes devolver todas las filas de miembros de la familia cuyo num_books_read sea mayor que 190? select * from family_members Where num_books_read > 190
5. **Resultado:** Una fila que muestra únicamente a Dave (con 200 libros leídos), excluyendo a Mary porque tiene 180.



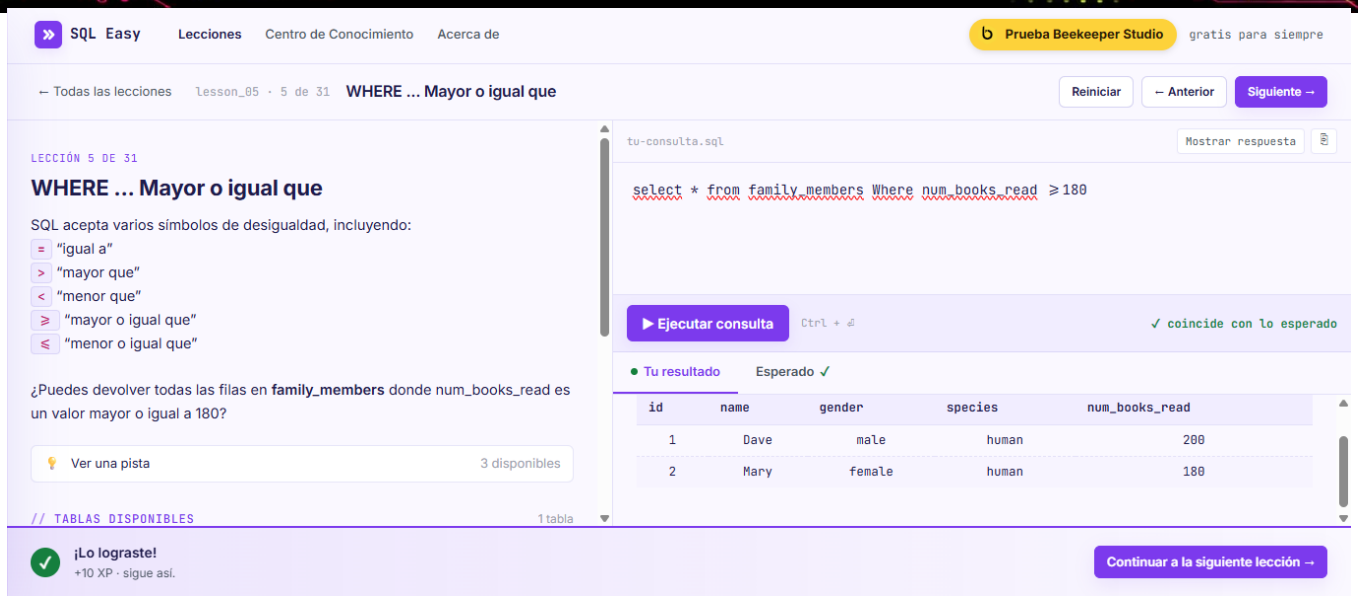


6. **Interpretación:** La base de datos compara el número de libros de cada integrante contra el número 190 y solo deja pasar a los que superan esa cantidad.
7. **Aprendizaje:** Comprendí cómo aplicar filtros basados en cantidades numéricas utilizando símbolos matemáticos básicos para comparar datos.
8. **Aplicación en el mundo real:** En una plataforma de ventas, esto sirve para filtrar y mostrar únicamente los productos cuyo precio sea mayor a $1,000 pesos.

Ejercicio 05: WHERE ... Mayor o igual que

1. **Número y nombre del ejercicio:** Lección 5: WHERE ... Mayor o igual que
2. **Objetivo del ejercicio:** Obtener los registros de los miembros que hayan leído 180 libros o más.
3. **Concepto SQL:** Uso del operador de comparación "mayor o igual que" (>=) para incluir el límite dentro de la búsqueda.
4. **Consulta ejecutada:** ¿Puedes devolver todas las filas en family_members donde num_books_read es un valor mayor o igual a 180? select * from family_members Where num_books_read >= 180;
5. **Resultado:** Dos filas con la información de Dave (200 libros) y Mary (180 libros).





6. **Interpretación:** A diferencia del ejercicio anterior, al usar el símbolo de mayor o igual, el número 180 sí entra en el resultado, por lo que Mary ahora sí aparece en pantalla.

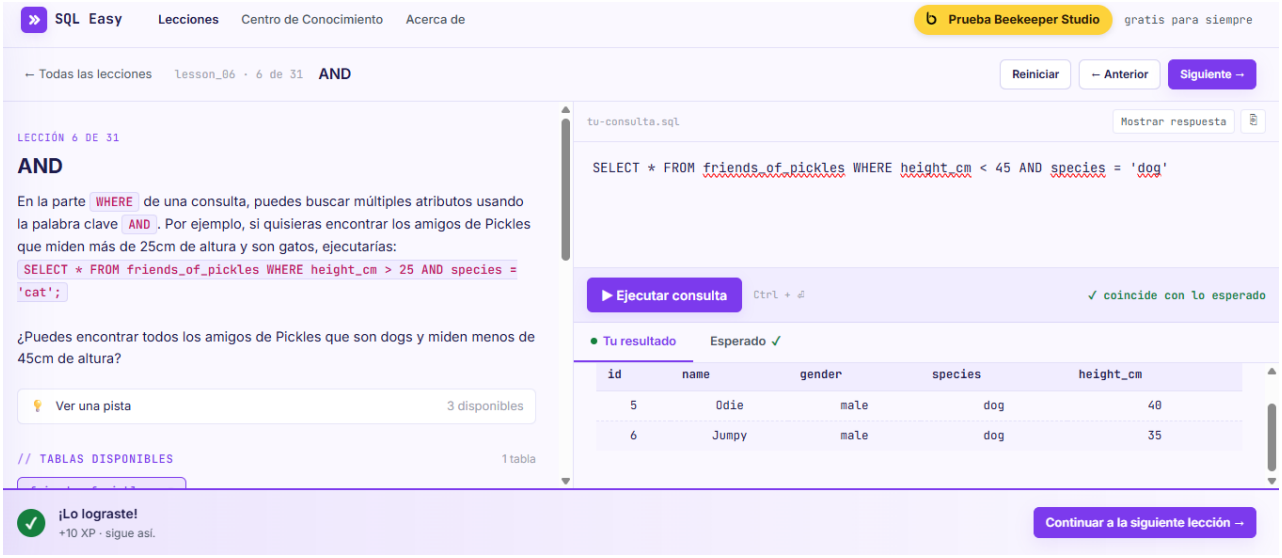
7. **Aprendizaje:** Entendí la importancia de incluir el signo de igual junto al de mayor que para asegurarnos de no dejar fuera datos que están exactamente en la línea del límite.
8. **Aplicación en el mundo real:** En un sistema escolar, se utiliza para filtrar a los alumnos cuya calificación final sea mayor o igual a 60, asegurando que nadie que haya obtenido exactamente 60 de calificación quede reprobado por error.

Para los ejercicios del 6 al 13 se observó y uso la siguiente tabla disponible:

id PK int	name char	gender char	species char	height_cm int
1	Dave	male	human	180
2	Mary	female	human	160
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

Ejercicio 06: AND (Múltiples condiciones obligatorias)

1. **Número y nombre del ejercicio:** Lección 6: AND
2. **Objetivo del ejercicio:** Encontrar a los amigos de Pickles que sean perros **y** que al mismo tiempo midan menos de 45 cm de altura.
3. **Concepto SQL:** Uso de la palabra clave AND dentro de la cláusula WHERE para exigir que se cumplan **dos o más condiciones simultáneamente**.
4. **Consulta ejecutada**: ¿Puedes encontrar todos los amigos de Pickles que son dogs y miden menos de 45cm de altura? SELECT * FROM friends_of_pickles WHERE height_cm < 45 AND species = 'dog';
5. **Resultado:** Los registros de Odie y Jumpy, que cumplen con ambas reglas a la vez.



6. **Interpretación:** La base de datos revisa cada registro y descarta a los que no cumplan con las dos reglas estrictas: si es perro, pero mide más de 45 cm, se queda fuera.

7. **Aprendizaje:** Comprendí cómo afinar las búsquedas exigiendo que coexistan dos o más filtros al mismo tiempo para obtener resultados más precisos.

8. **Aplicación en el mundo real:** En una tienda en línea, buscar un teléfono inteligente que sea de la marca "Samsung" y cuyo precio sea menor a $8,000 pesos al mismo tiempo.

Ejercicio 07: OR (Cualquiera de las condiciones)

1. **Número y nombre del ejercicio:** Lección 7: OR
2. **Objetivo del ejercicio:** Encontrar a los amigos que midan menos de 45 cm de altura **o** que pertenezcan a la especie perro.
3. **Concepto SQL:** Uso de la palabra clave OR para indicar que basta con que se cumpla al menos una de las opciones planteadas.
4. **Consulta ejecutada:** ¿Puedes encontrar todos los amigos de Pickles que son dogs o miden menos de 45cm de altura?select * from friends_of_pickles where height_cm < 45 or species = 'dog';
5. **Resultado:** Una lista más amplia que incluye a todos los que miden menos de 45 cm o son perros (como Fry, Leela, Odie, Jumpy y Sneakers).



6. **Interpretación:** A diferencia del AND, aquí el círculo de búsqueda se abre: si cumple la primera regla entra, y si cumple la segunda también entra.

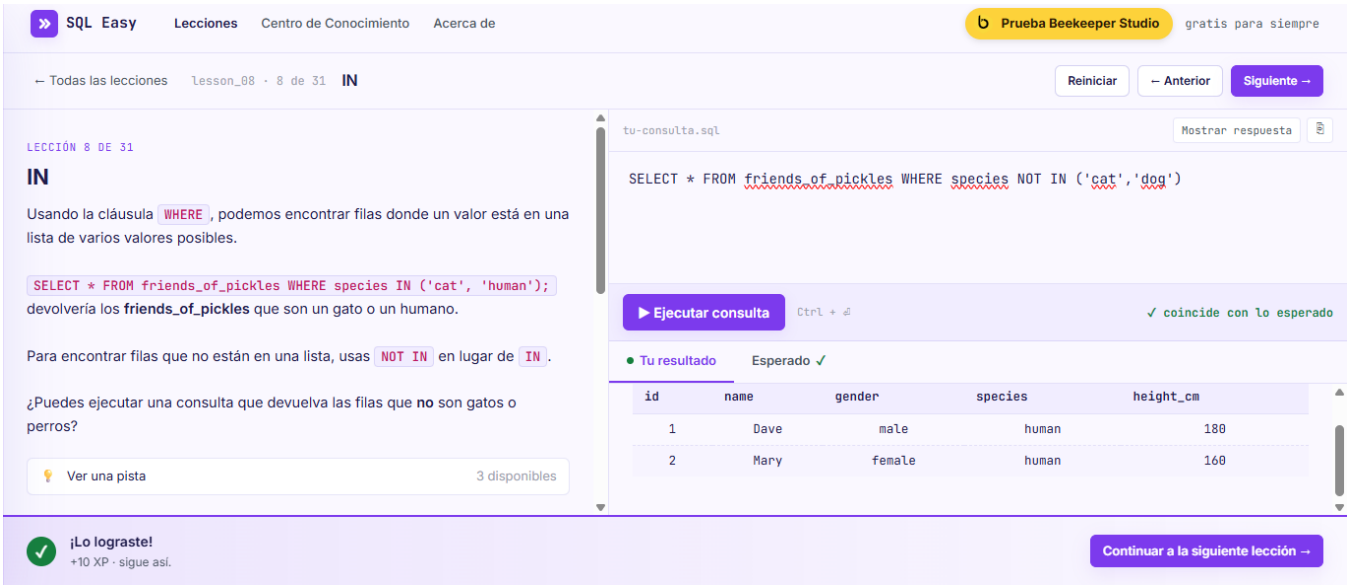
7. **Aprendizaje:** Entendí la diferencia clave entre AND y OR, aprendiendo a dar flexibilidad a los filtros cuando nos sirve cualquier opción disponible.

8. **Aplicación en el mundo real:** En un portal de viajes, buscar vuelos que salgan el sábado o que cuesten menos de $2,000 pesos (te mostrará las opciones que cumplan cualquiera de los dos requisitos).

Ejercicio 08: NOT IN (Excluir listas de valores)

1. **Número y nombre del ejercicio:** Lección 8: NOT IN
2. **Objetivo del ejercicio:** Mostrar las filas de los amigos de Pickles que no sean ni gatos ni perros.
3. **Concepto SQL:** Uso de NOT IN para excluir de la búsqueda a un grupo completo de valores especificados entre paréntesis.
4. **Consulta ejecutada:** ¿Puedes ejecutar una consulta que devuelva las filas que **no** son gatos o perros? SELECT * FROM friends_of_pickles WHERE species NOT IN ('cat', 'dog');

5. **Resultado:** Los registros de Dave y Mary (humanos), ya que se eliminaron de la vista todos los gatos y perros.



6. **Interpretación:** El sistema revisa la especie de cada integrante y descarta automáticamente a los que coincidan con la lista prohibida, dejando solo a los demás.

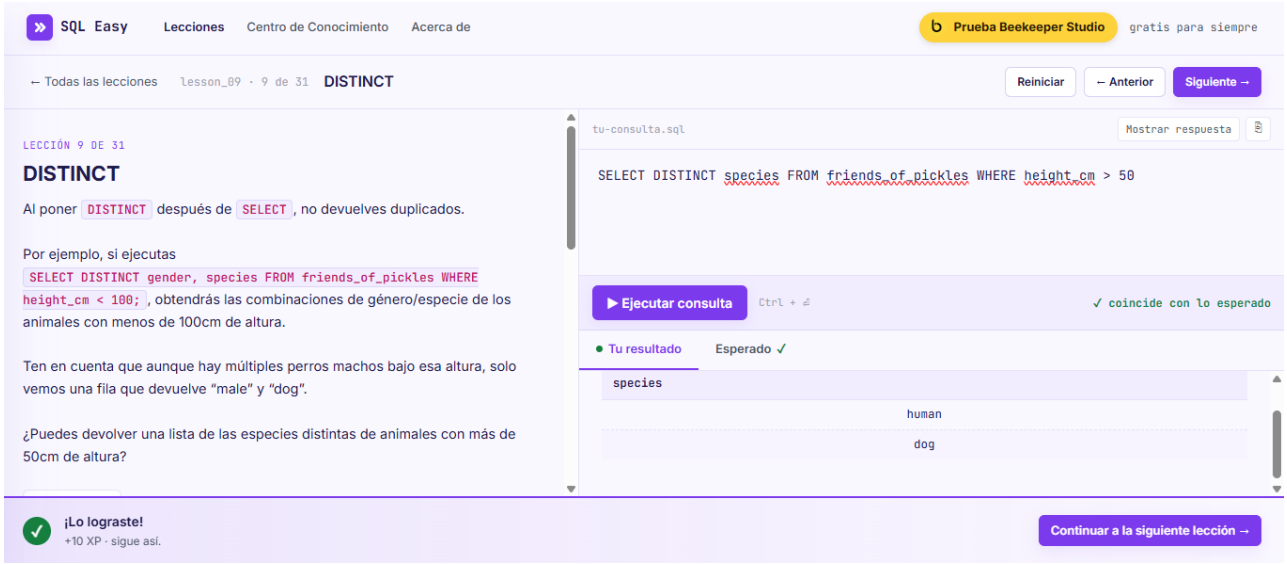
7. **Aprendizaje:** Aprendí a descartar múltiples opciones de un solo golpe sin tener que escribir varias líneas de exclusión por separado.

8. **Aplicación en el mundo real:** En el inventario de una tienda de ropa, filtrar y mostrar todos los productos que no pertenezcan a las categorías de calzado ni accesorios.

Ejercicio 09: DISTINCT (Eliminar duplicados)

1. **Número y nombre del ejercicio:** Lección 9: DISTINCT
2. **Objetivo del ejercicio:** Obtener una lista única con las especies de animales que miden más de 50 cm, evitando que aparezcan nombres repetidos.
3. **Concepto SQL:** Uso de DISTINCT después de SELECT para limpiar el resultado y mostrar únicamente valores únicos.

4. **Consulta ejecutada:** ¿Puedes devolver una lista de las especies distintas de animales con más de 50cm de altura? SELECT DISTINCT species FROM friends_of_pickles WHERE height_cm > 50;
5. **Resultado:** Una lista limpia con las especies (como human y dog) sin repeticiones.



6. **Interpretación:** Aunque varios registros compartan la misma especie, la base de datos los agrupa en una sola mención visual para entregar un reporte más corto.

7. **Aprendizaje:** Comprendí cómo evitar la saturación de información repetida cuando solo necesitamos conocer categorías o elementos únicos participan en los registros.

8. **Aplicación en el mundo real:** En una base de datos de clientes, de una tienda de ropa en línea se quiere obtener una lista única de los países desde donde se realizan compras, sin repetir el nombre del país por cada cliente que viva ahí:
   Juan compró una playera (País: **México**)
   Sofía compró un pantalón (País: **España**)
   Carlos compró una gorra (País: **México**)
   Andrea compró unos tenis (País: **Argentina**)
   Luis compró una chamarra (País: **España**)

   Si le pides a la computadora que te muestre los países de todos tus compradores sin usar DISTINCT, la lista saldría así:

México, España, México, Argentina, España (con repeticiones). Si aplicamos DISTINCT:

El resultado que devuelve la base de datos será **sin repetir:**

- **México**
- **España**
- **Argentina**

Ejercicio 10: ORDER BY DESC (Ordenar de mayor a menor)

1. **Número y nombre del ejercicio:** Lección 10: ORDER BY DESC
2. **Objetivo del ejercicio:** Organizar la tabla completa de los amigos de Pickles basándose en su altura en centímetros, colocándolos de mayor a menor estatura.
3. **Concepto SQL:** Uso de ORDER BY acompañado de la instrucción DESC (*Descending*) para ordenar los datos de forma descendente.
4. **Consulta ejecutada:** ¿Puedes ejecutar una consulta que ordene los **friends_of_pickles** por *height_cm* en orden descendente? select * from friends_of_pickles order by height_cm desc
5. **Resultado:** La lista acomodada perfectamente desde el animal más alto (40 cm) hasta el más bajo (25 cm).



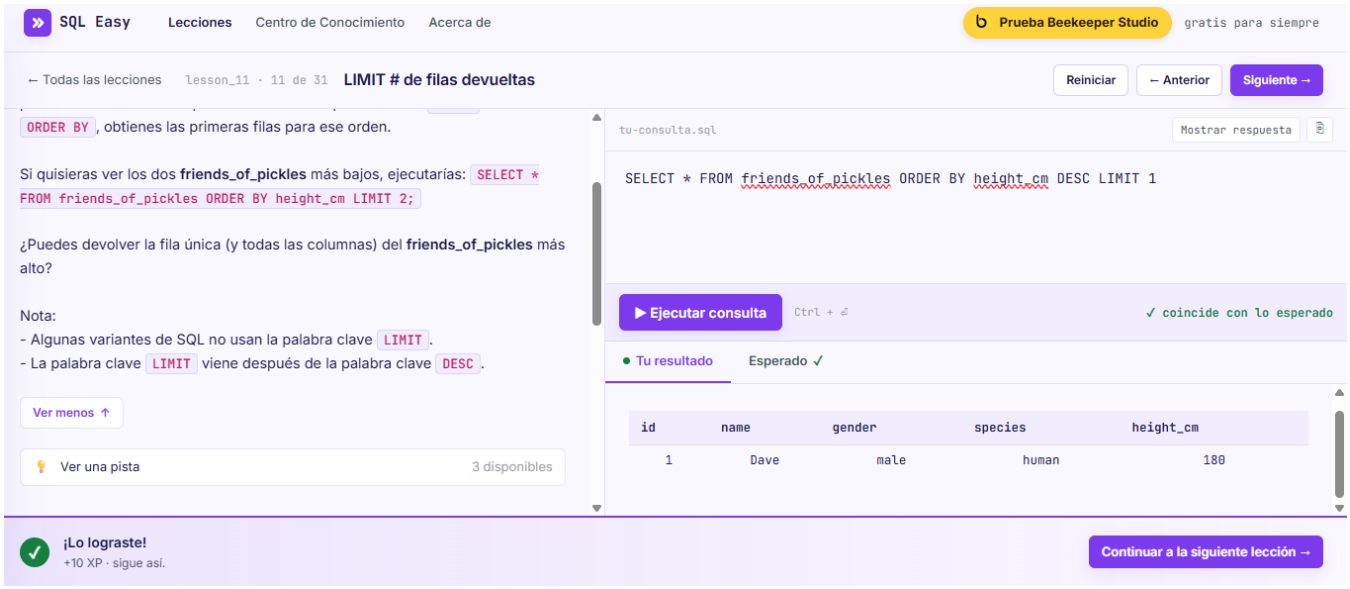
6. **Interpretación:** Los datos se reordenan de manera lógica para la lectura humana, facilitando identificar rápidamente quién encabeza la lista por su tamaño.

7. **Aprendizaje:** Aprendí a estructurar los datos para que salgan ordenados jerárquicamente y no de forma aleatoria.

8. **Aplicación en el mundo real:** En una tienda de comercio electrónico, ordenar un catálogo de productos por precio de mayor a menor para destacar los artículos más costosos o premium al inicio de la página.

Ejercicio 11: LIMIT (Limitar resultados)

1. **Número y nombre del ejercicio:** Lección 11: LIMIT # de filas devueltas
2. **Objetivo del ejercicio:** Devolver la fila única con la información del amigo de Pickles que tiene la mayor altura.
3. **Concepto SQL:** Uso de LIMIT para definir la cantidad máxima de filas que queremos que aparezcan en pantalla.
4. **Consulta ejecutada:** ¿Puedes devolver la fila única (y todas las columnas) del **friends_of_pickles** más alto? SELECT * FROM friends_of_pickles ORDER BY height_cm DESC LIMIT 1
5. **Resultado:** Una sola fila correspondiente a Dave, quien mide 180 cm de altura.



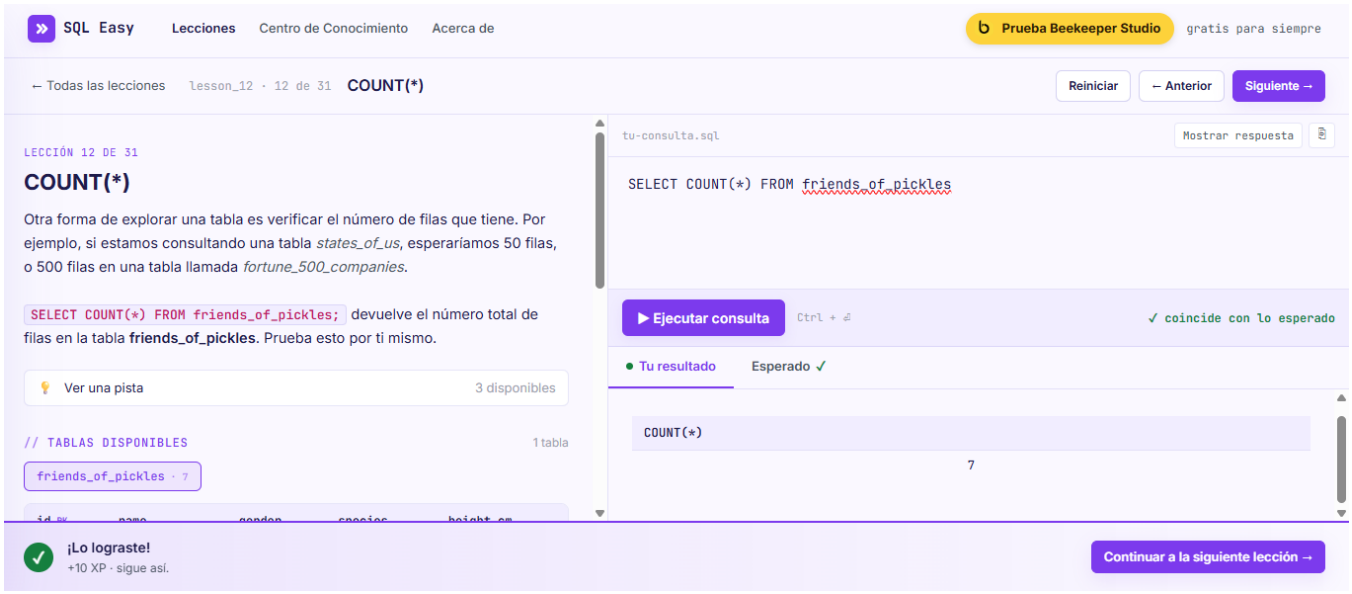
Resultado: Una sola fila correspondiente a Dave, quien mide 180 cm de altura.

6. **Interpretación:** La consulta ordena a todos de mayor a menor estatura y la instrucción LIMIT 1 le indica a la base de datos que se detenga inmediatamente al encontrar al primer lugar.
7. **Aprendizaje:** Entendí cómo extraer únicamente los primeros puestos de un listado ordenado sin necesidad de ver toda la base de datos.
8. **Aplicación en el mundo real:** El Gerente de almacén e Inventario quiere identificar rápidamente sus artículos más vendidos para surtir stock. Ordena la lista de productos por unidades vendidas de mayor a menor y aplica un límite (**LIMIT 10)**.

Ejercicio 12: COUNT(*) (Contar filas totales)

1. **Número y nombre del ejercicio:** Lección 12: COUNT(*)
2. **Objetivo del ejercicio:** Contar el número total de registros existentes en la tabla friends_of_pickles.
3. **Concepto SQL:** Uso de la función de agregación COUNT(*) para obtener la cantidad total de filas que conforman una tabla.
4. **Consulta ejecutada:** devuelve el número total de filas en la tabla friends_of_pickles. SELECT COUNT(*) FROM friends_of_pickles;

5. **Resultado:** El número 7

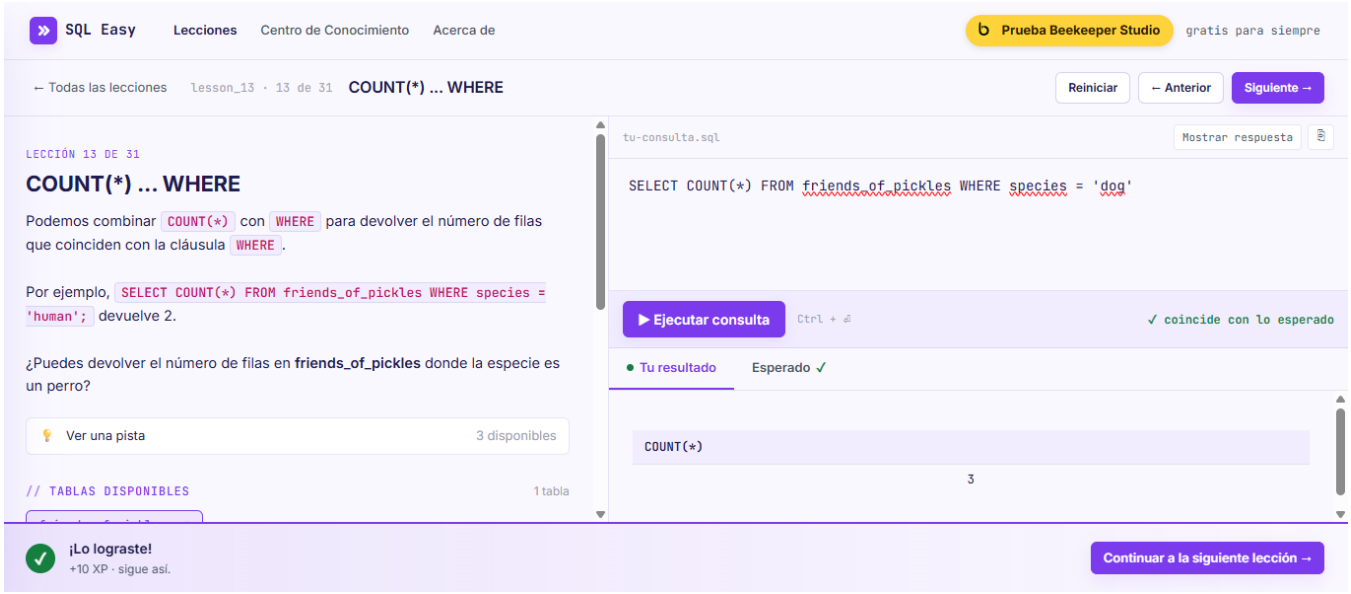


6. **Interpretación:** En lugar de mostrar los nombres o características de los integrantes, la base de datos simplemente realiza un conteo rápido de cuántos registros hay almacenados.
7. **Aprendizaje:** Comprendí la diferencia entre pedirle a SQL que me devuelva datos detallados y pedirle una cifra total sumada de elementos.
8. **Aplicación en el mundo real:** En una tienda sirve para conocer rápidamente cuántos clientes registrados hay en un sistema.

Ejercicio 13: COUNT(*) ... WHERE (Contar con condiciones)

1. **Número y nombre del ejercicio:** Lección 13: COUNT(*) ... WHERE
2. **Objetivo del ejercicio:** Saber cuántos amigos de Pickles pertenecen a la especie perro (dog).
3. **Concepto SQL:** Combinación de la función COUNT(*) con el filtro WHERE para contar solo los registros que cumplen una condición.
4. **Consulta ejecutada:** ¿Puedes devolver el número de filas en **friends_of_pickles** donde la especie es un perro? SELECT COUNT(*) FROM friends_of_pickles WHERE species = 'dog';

5. **Resultado:** El número 3.



6. **Interpretación:** La base de datos busca únicamente las filas donde la columna species sea "dog" y cuenta solo esos casos específicos, ignorando a los demás.
7. **Aprendizaje:** Aprendí a realizar censos o conteos focalizados sobre un segmento específico de la información.
8. **Aplicación en el mundo real:** Contar cuántos usuarios con cuenta "Premium" existen en una plataforma web.

Para los ejercicios del 14 al 16 se observó y uso la siguiente tabla disponible:



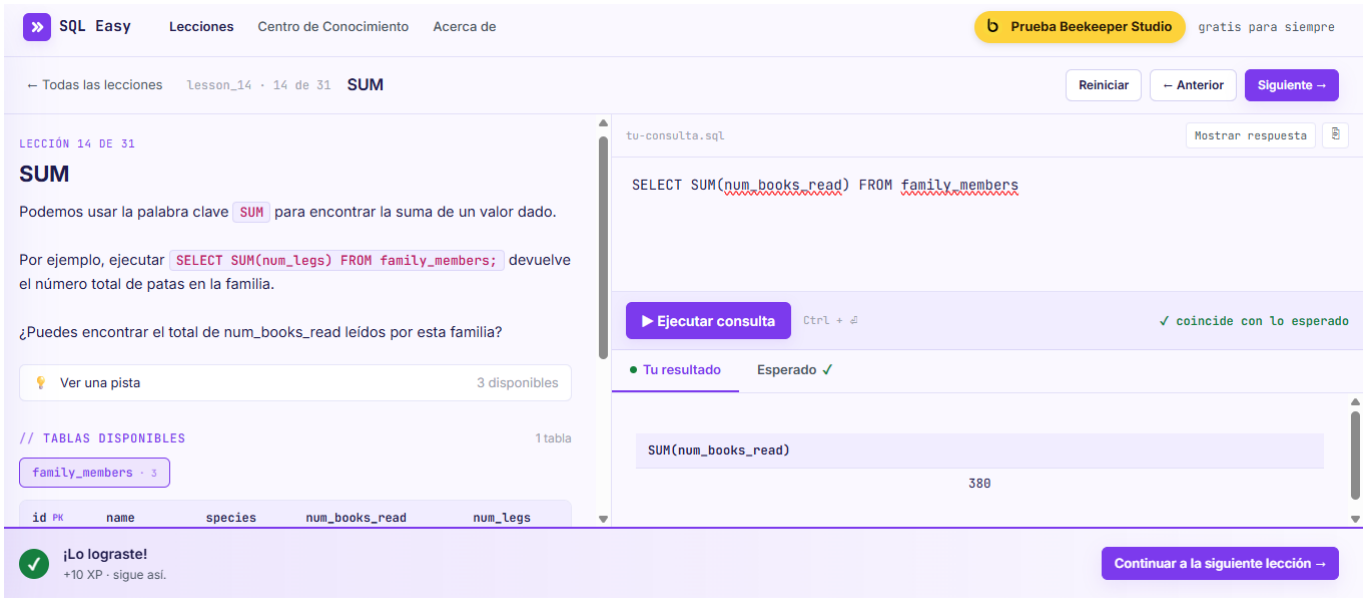
// TABLAS DISPONIBLES — 1 tabla

family_members · 3

id PK int	name char	species char	num_books_read int	num_legs int
1	Dave	human	200	2
2	Mary	human	180	2
3	Pickles	dog	0	4

Ejercicio 14: SUM (Sumar valores numéricos)

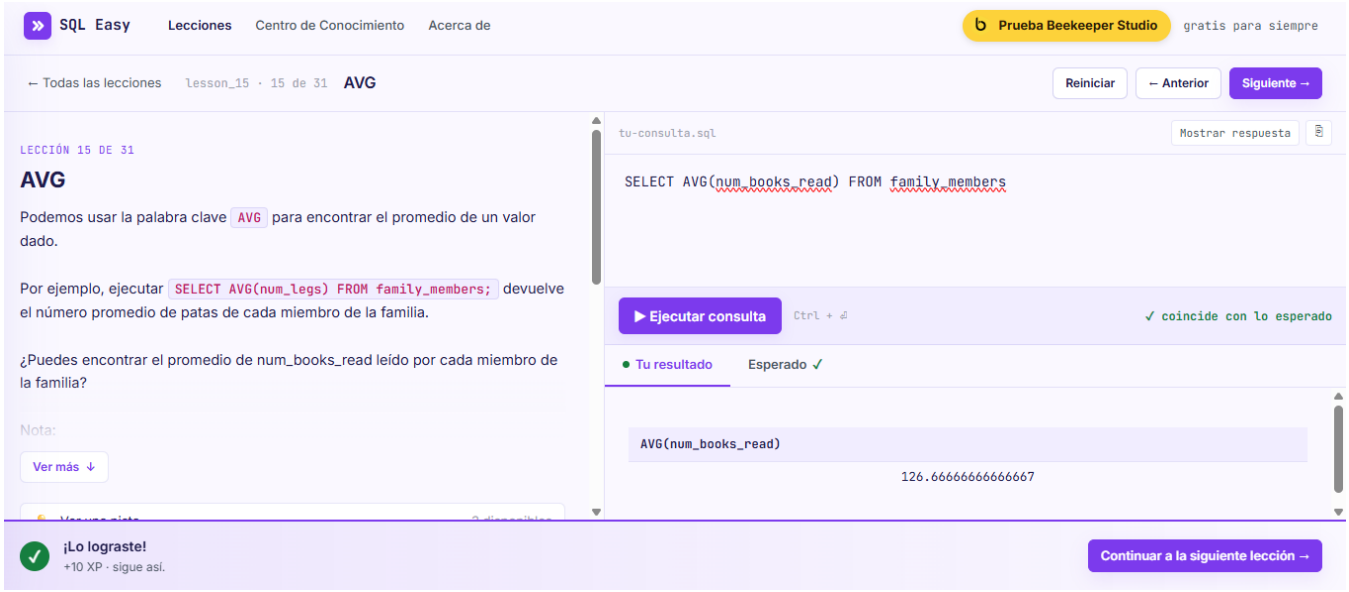
1. **Número y nombre del ejercicio:** Lección 14: SUM
2. **Objetivo del ejercicio:** Calcular la suma total de libros leídos por todos los miembros de la familia.
3. **Concepto SQL:** Uso de la función SUM() para sumar automáticamente todos los valores de una columna numérica.
4. **Consulta ejecutada:** ¿Puedes encontrar el total de num_books_read leídos por esta familia? SELECT SUM(num_books_read) FROM family_members;
5. **Resultado:** El número 380.



6. **Interpretación:** SQL toma cada número de la columna num_books_read (200 + 180 + 0) y realiza la operación matemática en un solo paso.
7. **Aprendizaje:** Comprendí cómo realizar operaciones de suma de forma instantánea sobre un grupo entero de datos numéricos sin hacerlo manualmente.
8. **Aplicación en el mundo real:** Calcular el total de ventas acumuladas en dinero al final del día en un restaurante o tienda.

Ejercicio 15: AVG (Calcular el promedio)

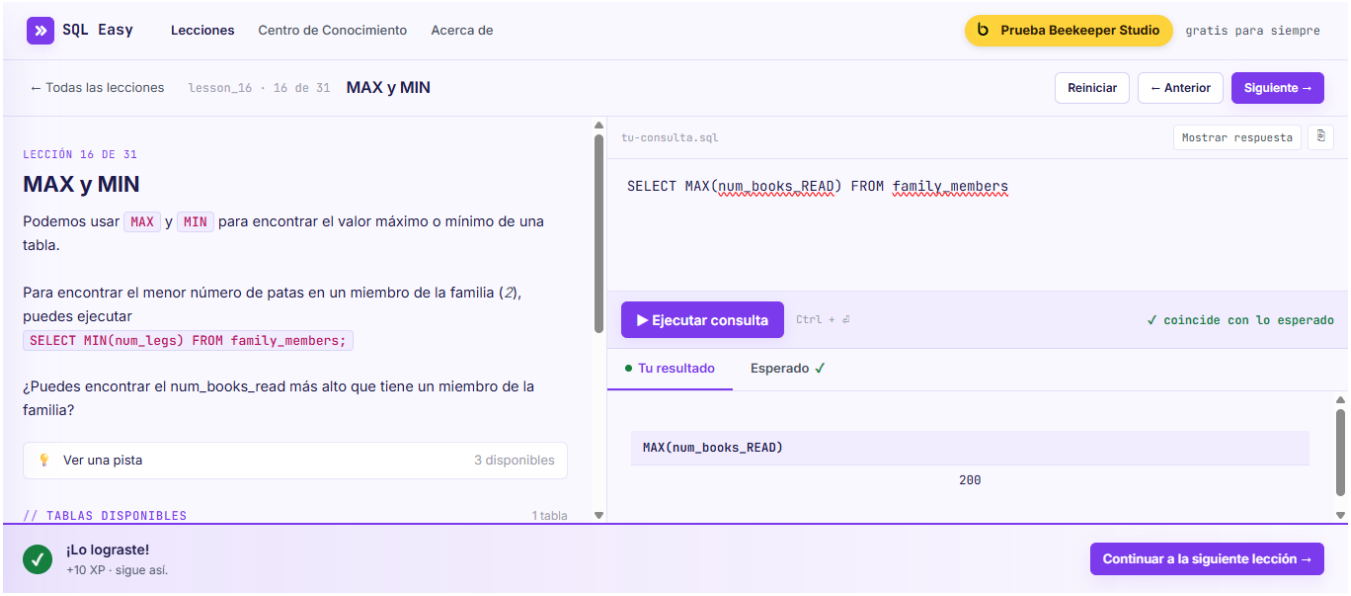
1. **Número y nombre del ejercicio:** Lección 15: AVG
2. **Objetivo del ejercicio:** Obtener la cantidad promedio de libros leídos por cada miembro de la familia.
3. **Concepto SQL:** Uso de la función AVG() (*Average*) para calcular el promedio de una columna de datos numéricos.
4. **Consulta ejecutada:** ¿Puedes encontrar el promedio de num_books_read leído por cada miembro de la familia? SELECT AVG(num_books_read) FROM family_members;
5. **Resultado:** El número 126.66666666666667.



6. **Interpretación:** La base de datos suma el total de libros leídos (380) y lo divide entre los 3 integrantes de la familia para obtener el valor promedio por persona.
7. **Aprendizaje:** Aprendí a obtener indicadores estadísticos básicos como el promedio mediante una orden.
8. **Aplicación en el mundo real:** Un contador de una empresa calcula el costo de operación diario para presupuestos futuros usando **AVG (gasto_diario)**. El resultado le indica cuánto dinero en promedio gasta la empresa por día en insumos y servicios básicos.

Ejercicio 16: MAX y MIN (Valor máximo y mínimo)

1. **Número y nombre del ejercicio:** Lección 16: MAX y MIN
2. **Objetivo del ejercicio:** Encontrar el número máximo de libros leídos por cualquier miembro de la familia.
3. **Concepto SQL:** Uso de la función MAX() para identificar el valor más alto dentro de una columna de datos numéricos.
4. **Consulta ejecutada:** ¿Puedes encontrar el num_books_read más alto que tiene un miembro de la familia? SELECT MAX(num_books_read) FROM family_members;
5. **Resultado:** El número 200.



6. **Interpretación:** La base de datos examina todos los registros de la columna y devuelve únicamente la cifra tope, que en este caso le pertenece a Dave.



7. **Aprendizaje:** Comprendí cómo localizar valores superiores e inferiores de manera directa sin tener que ordenar toda la lista.

8. **Aplicación en el mundo real:** En una empresa, encontrar el precio del producto más caro en inventario o el récord de ventas registrado en un día.

Para el ejercicio 17 se observó y uso la siguiente tabla disponible:

friends_of_pickles · 7

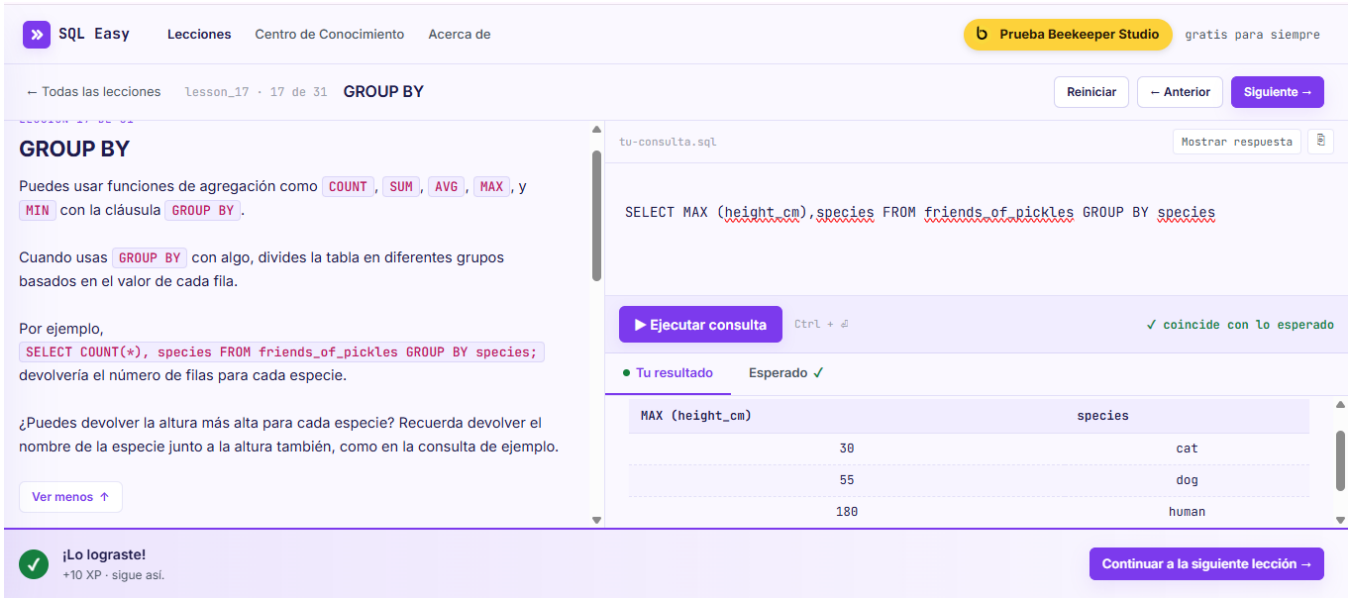
id PK int	name char	gender char	species char	height_cm int
1	Dave	male	human	180
2	Mary	female	human	160
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

Ejercicio 17: GROUP BY (Agrupar información)

1. **Número y nombre del ejercicio:** Lección 17: GROUP BY
2. **Objetivo del ejercicio:** Mostrar la altura máxima registrada para cada una de las diferentes especies de la tabla.
3. **Concepto SQL:** Uso de GROUP BY para clasificar las filas en categorías o subgrupos antes de aplicarles una función de cálculo (como MAX).

4. **Consulta ejecutada:** ¿Puedes devolver la altura más alta para cada especie? Recuerda devolver el nombre de la especie junto a la altura también SELECT MAX(height_cm), species FROM friends_of_pickles GROUP BY species;

5. **Resultado:** Una tabla que muestra la altura tope separada por categoría: gatos (30 cm), perros (55 cm) y humanos (180 cm).



6. **Interpretación:** SQL organiza primero a los miembros por su especie y luego calcula el valor máximo de altura de forma independiente para cada grupo.

7. **Aprendizaje:** Aprendí a segmentar por categorías para obtener métricas individuales sin revolver la información de distintos grupos.

8. **Aplicación en el mundo real:** En una cadena de tiendas, calcular las ventas totales agrupadas por cada sucursal o ciudad.

Para el ejercicio 18 se observó y uso la siguiente tabla disponible:



// TABLAS DISPONIBLES 1 tabla

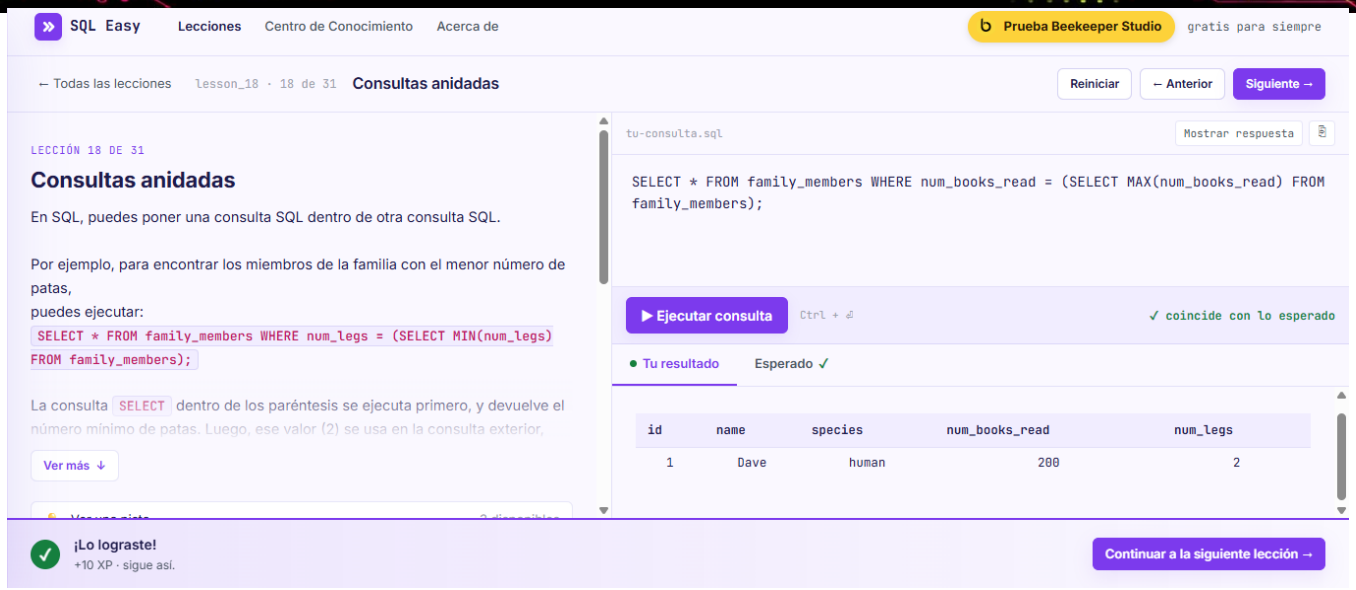
family_members · 3

id PK int	name char	species char	num_books_read int	num_legs int
1	Dave	human	200	2
2	Mary	human	180	2
3	Pickles	dog	0	4

Ejercicio 18: Consultas anidadas (Subconsultas)

1. **Número y nombre del ejercicio:** Lección 18: Consultas anidadas
2. **Objetivo del ejercicio:** Devolver la información completa del miembro de la familia que haya leído el mayor número de libros.
3. **Concepto SQL:** Uso de una subconsulta (una orden SELECT dentro de paréntesis) para calcular un valor intermedio y usarlo como filtro en la consulta principal.
4. **Consulta ejecutada:** ¿Puedes devolver los miembros de la familia que tienen el num_books_read más alto? SELECT * FROM family_members WHERE num_books_read = (SELECT MAX(num_books_read) FROM family_members);
5. **Resultado:** La fila entera de Dave (ID 1, género male, especie human, 200 libros leídos).





6. **Interpretación:** Primero, el sistema resuelve los paréntesis averiguando cuál es el récord máximo (200) y luego busca a la persona que coincide con esa cifra exacta.

7. **Aprendizaje:** Comprendí cómo encadenar dos preguntas a la base de datos en una sola línea para obtener respuestas complejas.

8. **Aplicación en el mundo real:** En Recursos Humanos, buscar al empleado que tenga el número de ventas más alto para otorgarle un reconocimiento de desempeño.

Para el ejercicio 19 se observó y uso la siguiente tabla disponible:



// TABLAS DISPONIBLES 1 tabla

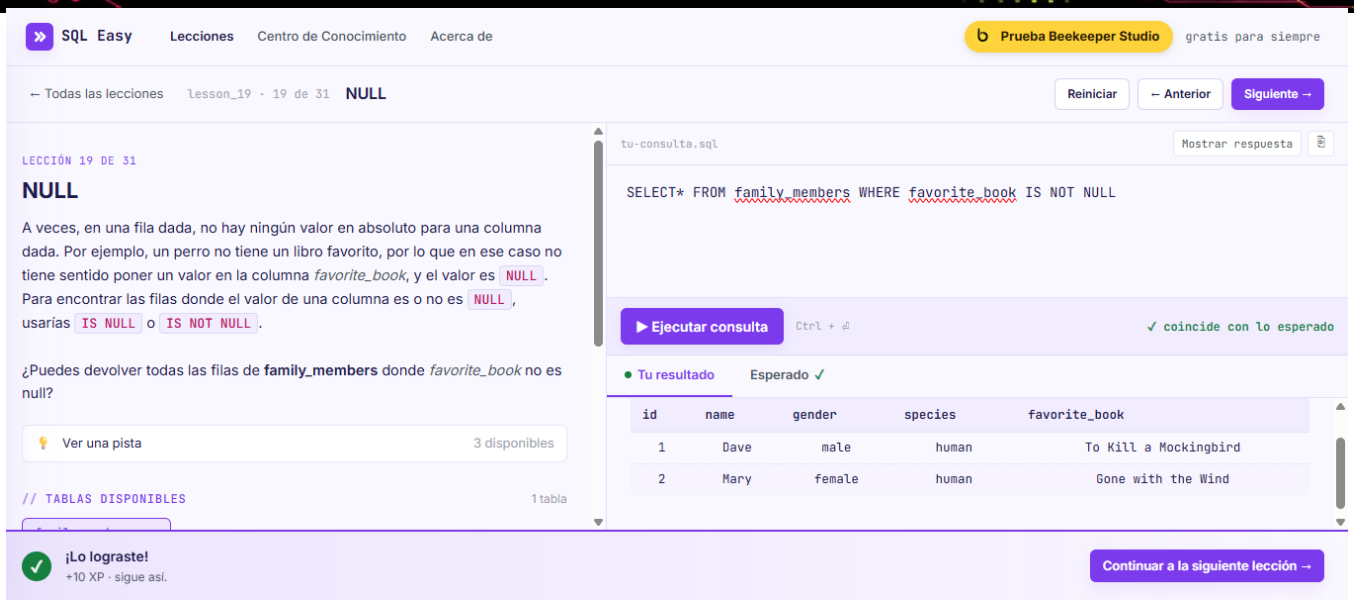
family_members · 3

id PK int	name char	gender char	species char	favorite_book char
1	Dave	male	human	To Kill a Mockingbird
2	Mary	female	human	Gone with the Wind
3	Pickles	male	dog	*NULL*

Ejercicio 19: NULL (Datos faltantes o vacíos)

1. **Número y nombre del ejercicio:** Lección 19: NULL
2. **Objetivo del ejercicio:** Mostrar únicamente los miembros de la familia que sí tengan un libro favorito registrado (que no esté vacío).
3. **Concepto SQL:** Uso de la condición IS NOT NULL para filtrar y descartar las celdas o registros a los que les falta información.
4. **Consulta ejecutada:** ¿Puedes devolver todas las filas de **family_members** donde *favorite_book* no es null? SELECT * FROM family_members WHERE favorite_book IS NOT NULL
5. **Resultado:** Las filas de Dave y Mary, excluyendo a la mascota porque no tiene un libro favorito asignado.





6. **Interpretación:** La base de datos revisa la columna favorite_book y omite todas las filas que no contengan texto o datos reales.

7. **Aprendizaje:** Aprendí a trabajar únicamente con registros que tengan expedientes o campos completos.

8. **Aplicación en el mundo real:** En una campaña de mercadotecnia por correo, filtrar la lista de clientes para incluir solo a aquellos que sí tengan registrado un correo electrónico válido.

Para el ejercicio 20 se observó y uso la siguiente tabla disponible:




```
// TABLAS DISPONIBLES

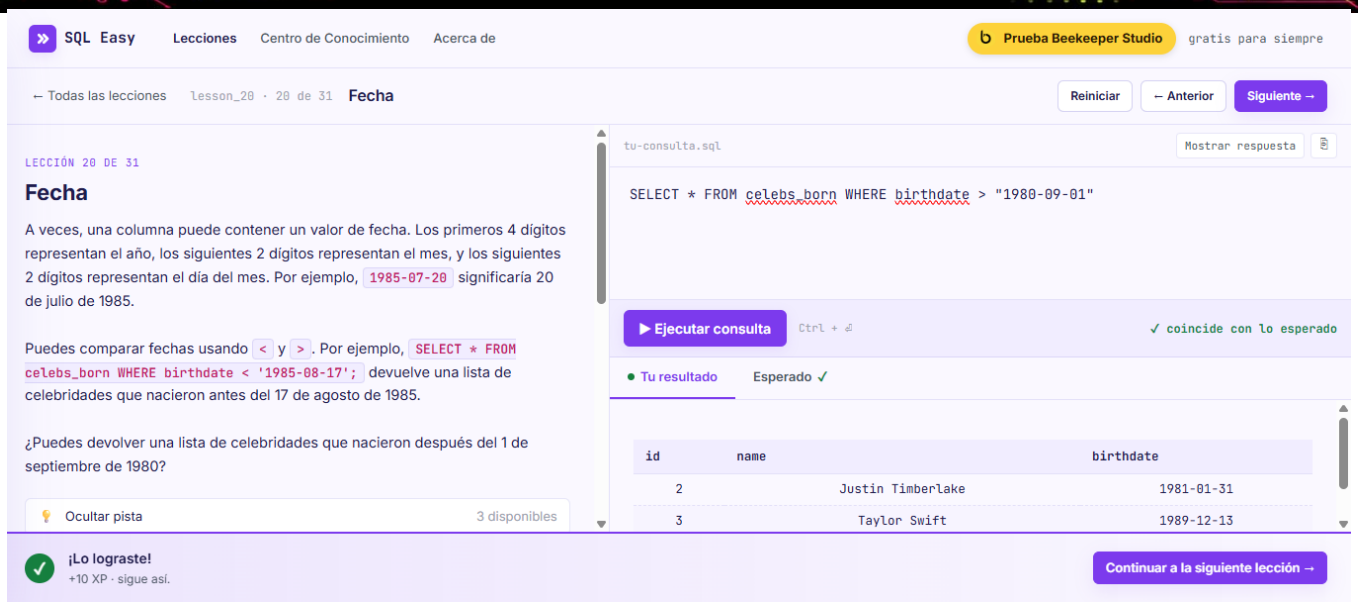
celebs_born · 3
```

id PK int	name char	birthdate date
1	Michael Jordan	1963-02-17
2	Justin Timberlake	1981-01-31
3	Taylor Swift	1989-12-13

Ejercicio 20: Fecha (Filtros por tiempo)

1. **Número y nombre del ejercicio:** Lección 20: Fecha
2. **Objetivo del ejercicio:** Obtener la lista de celebridades que nacieron después del 1 de septiembre de 1980.
3. **Concepto SQL:** Comparación de campos de tipo fecha utilizando operadores de desigualdad (>) y el formato estándar AAAA-MM-DD.
4. **Consulta ejecutada:** ¿Puedes devolver una lista de celebridades que nacieron después del 1 de septiembre de 1980? SELECT * FROM celebs_born WHERE birthdate > "1980-09-01";
5. **Resultado:** Las filas correspondientes a Justin Timberlake (nacido en 1981) y Taylor Swift (nacida en 1989).





6. **Interpretación:** El sistema evalúa el orden cronológico de las fechas de nacimiento e incluye únicamente a las personas que nacieron en días o años posteriores al límite establecido.
7. **Aprendizaje:** Comprendí que las fechas en SQL se pueden filtrar con el mismo método que los números normales para acotar períodos de tiempo.
8. **Aplicación en el mundo real:** En el departamento de Finanzas, consultar todas las facturas o transacciones realizadas después de cierta fecha para cerrar el mes contable.

Para el ejercicio 21 se observaron y usaron las siguientes tablas disponibles:





// TABLAS DISPONIBLES

character · 4 | character_tv_show · 4 | character_actor · 4

id PK int	character_id int → character.id	tv_show_name char
1	4	Buffy the Vampire Slayer
2	3	How I Met Your Mother
3	2	How I Met Your Mother
4	1	Doogie Howser, M.D.

// TABLAS DISPONIBLES 3 tablas

character · 4 | character_tv_show · 4 | character_actor · 4

id PK int	character_id int → character.id	actor_name char
1	4	Alyson Hannigan
2	3	Alyson Hannigan
3	2	Neil Patrick Harris
4	1	Neil Patrick Harris

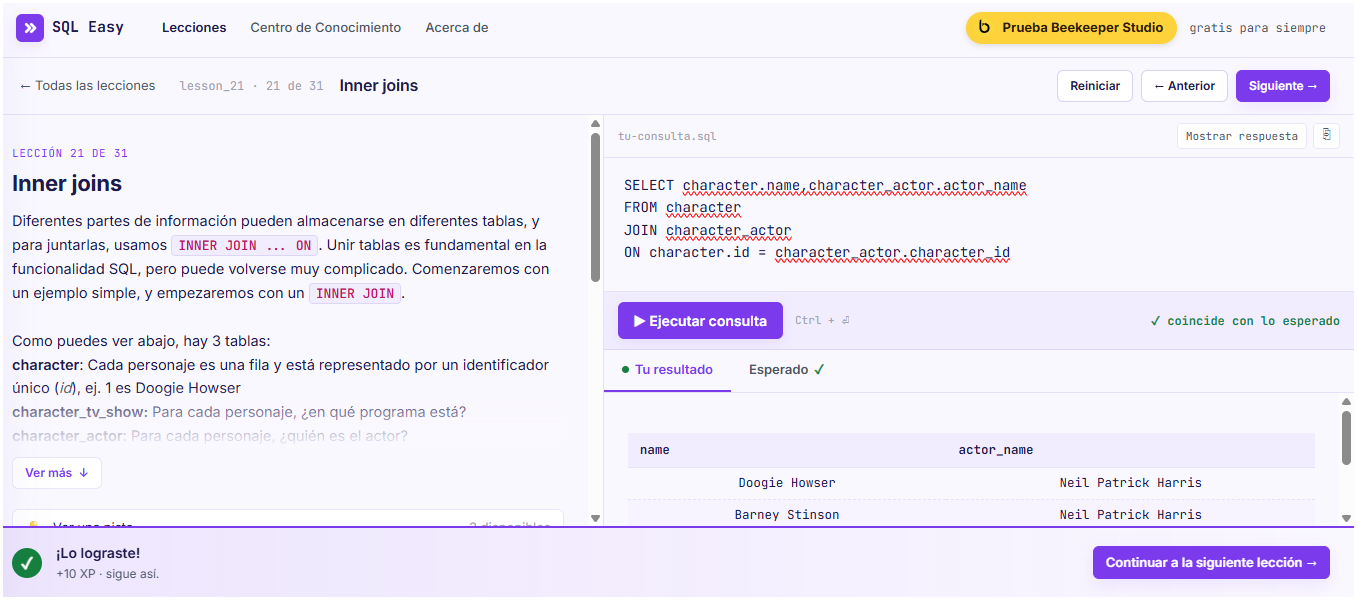
Ejercicio 21: Inner joins (Unir tablas relacionales)

1. **Número y nombre del ejercicio:** Lección 21: Inner joins
2. **Objetivo del ejercicio:** Relacionar la tabla de personajes con la tabla de actores para mostrar qué actor interpreta a cada personaje en pantalla.
3. **Concepto SQL:** Uso de INNER JOIN combinado con ON para cruzar la información de dos tablas distintas mediante un código o identificador en común (id).

4. **Consulta ejecutada:**
   ¿Puedes usar un inner join para emparejar cada nombre de personaje con el actor que lo interpreta? SELECT character.name, character_actor.actor_name

FROM character JOIN character_actor ON character.id = character_actor.character_id

5. **Resultado:** Una lista limpia que vincula a Doogie Howser y Barney Stinson con su actor real (Neil Patrick Harris).



6. **Interpretación:** La base de datos busca las coincidencias de código entre ambas tablas y las combina en una sola vista coherente que une al personaje con su actor.

7. **Aprendizaje:** Comprendí cómo juntar piezas de información que están guardadas en tablas diferentes.

8. **Aplicación en el mundo real:** En una tienda en línea, cruzar la tabla de "Pedidos" con la tabla de "Clientes" para mostrar el nombre del comprador al lado de cada número de pedido.

Para los ejercicios del 22 al 26 se observaron y usaron las siguientes tablas disponibles:



// TABLAS DISPONIBLES

character · 4 | tv_show · 3 | character_tv_show · 4 | actor · 2 | character_actor · 4

id PK int	name char
1	Doogie Howser
2	Barney Stinson
3	Lily Aldrin
4	Willow Rosenberg

// TABLAS DISPONIBLES

character · 4 | tv_show · 3 | character_tv_show · 4 | actor · 2 | character_actor · 4

id PK int	name char
1	Buffy the Vampire Slayer
2	How I Met Your Mother
3	Doogie Howser, M.D.



// TABLAS DISPONIBLES

character · 4 | tv_show · 3 | character_tv_show · 4 | actor · 2 | character_actor · 4

id PK int	character_id int → character.id	tv_show_id int → tv_show.id
1	1	3
2	2	2
3	3	2
4	4	1

// TABLAS DISPONIBLES

character · 4 | tv_show · 3 | character_tv_show · 4 | actor · 2 | character_actor · 4

id PK int	name char
1	Alyson Hannigan
2	Neil Patrick Harris



// TABLAS DISPONIBLES

character · 4 | tv_show · 3 | character_tv_show · 4 | actor · 2

character_actor · 4

id PK int	character_id int → character.id	actor_id int → actor.id
1	1	2
2	2	2
3	3	1
4	4	1

Ejercicio 22: Múltiples joins

1. **Número y nombre del ejercicio:** Lección 22: Múltiples joins
2. **Objetivo del ejercicio:** Vincular tres tablas diferentes para mostrar los nombres de los personajes junto con los nombres de sus respectivos actores sin duplicar información estructural.
3. **Concepto SQL:** Uso de **múltiples INNER JOIN** en una sola consulta para conectar una tabla principal con dos tablas relacionadas de forma sucesiva.
4. **Consulta ejecutada**: ¿Puedes usar dos joins para emparejar el nombre de cada personaje con el actor que los interpreta? SELECT character.name, actor.name FROM character INNER JOIN character_actor ON character.id = character_actor.character_id INNER JOIN actor ON character_actor.actor_id = actor.id
5. **Resultado:** Una lista organizada que relaciona directamente el nombre del personaje con el nombre del actor (por ejemplo, Doogie Howser con Neil Patrick Harris).



6. **Interpretación:** La base de datos cruza primero la tabla de personajes con la tabla intermedia de relaciones, y luego salta a la tabla final de actores para unificar los datos en una sola vista.

7. **Aprendizaje:** Comprendí cómo escalar las consultas relacionales cuando un sistema requiere unir más de dos tablas para entregar un reporte completo.

8. **Aplicación en el mundo real:** En un sistema de ventas corporativo, unir la tabla de "Clientes", la tabla de "Pedidos" y la tabla de "Métodos de Pago" para ver qué cliente usó qué tarjeta en cada compra.

Ejercicio 23: Joins con WHERE

1. **Número y nombre del ejercicio:** Lección 23: Joins con WHERE
2. **Objetivo del ejercicio:** Obtener una lista cruzada de personajes y programas de TV excluyendo explícitamente a un programa específico y a un personaje determinado.
3. **Concepto SQL:** Combinación de múltiples uniones (INNER JOIN) con filtros restrictivos utilizando la cláusula WHERE y operadores de desigualdad (<>).

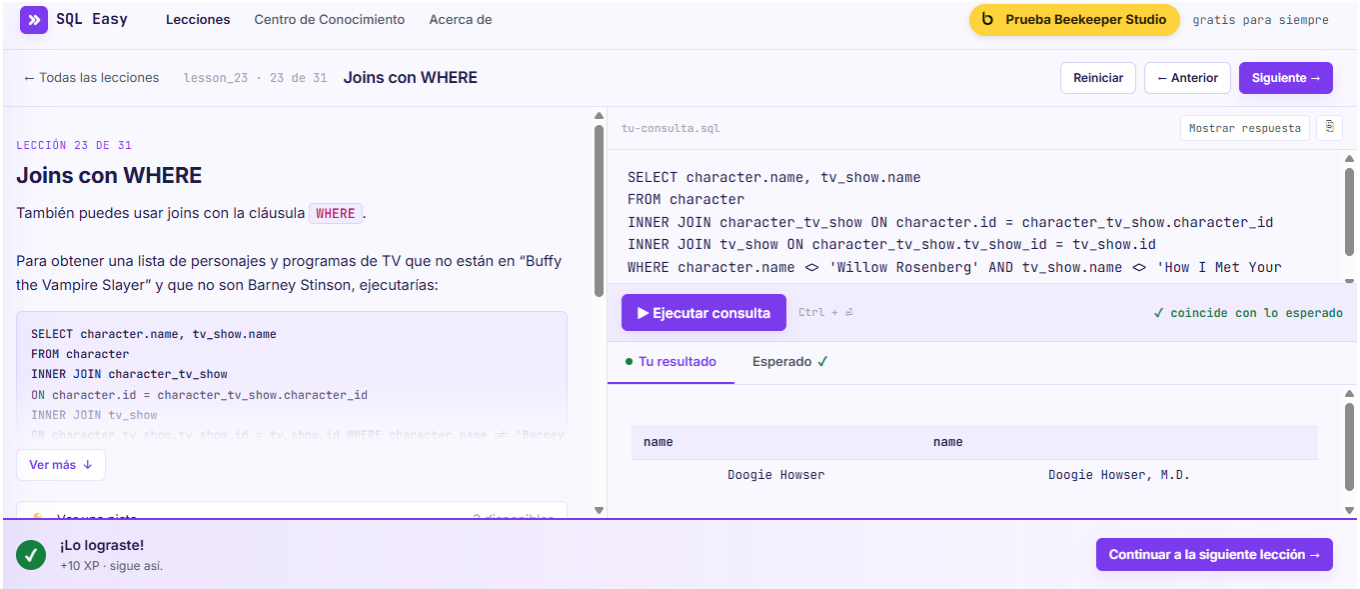
4. **Consulta ejecutada:** ¿Puedes devolver una lista de personajes y programas de TV que no se llamen “Willow Rosenberg” y que no estén en el programa “How I Met Your Mother”? SELECT character.name, tv_show.name FROM character

INNER JOIN character_tv_show ON character.id = character_tv_show.character_id

INNER JOIN tv_show ON character_tv_show.tv_show_id = tv_show.id

WHERE character.name <> 'Willow Rosenberg' AND tv_show.name <> 'How I Met Your Mother';

5. **Resultado:** Los registros que cumplen con el cruce de tablas, pero descartan por completo a los elementos prohibidos en el filtro.



6. **Interpretación:** Primero se unen las tablas para armar el universo completo de relaciones y, posteriormente, el filtro WHERE limpia el resultado eliminando las filas no deseadas.

7. **Aprendizaje:** Aprendí a afinar las consultas de tablas múltiples aplicando restricciones exactas sobre los datos combinados.

8. **Aplicación en el mundo real:** En una base de datos de inventario, unir proveedores y productos, pero filtrar para mostrar solo aquellos artículos que no pertenezcan a una marca descontinuada.

Ejercicio 24: Left joins

1. **Número y nombre del ejercicio:** Lección 24: Left joins

2. **Objetivo del ejercicio:** Obtener la lista completa de todos los personajes, incluso si algunos de ellos no tienen datos asociados en las tablas secundarias de actores.
3. **Concepto SQL:** Uso de LEFT JOIN (unión izquierda), el cual devuelve **todos** los registros de la primera tabla y rellena con NULL aquellos campos donde no encuentre coincidencias en la segunda tabla.
4. **Consulta ejecutada**: ¿Puedes usar left joins para emparejar nombres de personajes con los actores que los interpretan?

SELECT character.name, actor.name FROM character

LEFT JOIN character_actor ON character.id = character_actor.character_id

LEFT JOIN actor ON character_actor.actor_id = actor.id;

5. **Resultado:** El listado completo de personajes de la tabla izquierda acompañados de sus actores, mostrando valores vacíos (NULL) en caso de que un personaje no tenga actor asignado.



6. **Interpretación:** A diferencia del INNER JOIN (que descarta lo que no coincide), el LEFT JOIN protege y respeta la tabla principal asegurando que ningún registro se quede fuera del reporte.

7. **Aprendizaje:** Comprendí la importancia de conservar la visibilidad de los registros principales, aunque existan vacíos de información en las relaciones secundarias.

8. **Aplicación en el mundo real:** En recursos humanos, listar a **todos** los empleados de la empresa, mostrando al lado el curso que tomaron, asegurando que aparezcan también los empleados que aún no se han inscrito a ninguno.

Ejercicio 25: Alias de tabla

1. **Número y nombre del ejercicio:** Lección 25: Alias de tabla
2. **Objetivo del ejercicio:** Simplificar una consulta larga de múltiples uniones utilizando nombres cortos o apodos (*alias*) para las tablas involucradas.
3. **Concepto SQL:** Uso de la palabra clave AS para renombrar temporalmente las tablas dentro de la consulta (por ejemplo, character AS c).
4. **Consulta ejecutada**: ¿Puedes usar left joins para emparejar nombres de personajes con los actores que los interpretan, y usar alias para hacer la consulta más corta? Los alias para character, character_actor, y actor deben ser c, ca, y a.

```
SELECT c.name, a.name FROM character AS c
LEFT JOIN character_actor AS ca ON c.id = ca.character_id
LEFT JOIN actor AS a ON ca.actor_id = a.id
```

5. **Resultado:** El mismo resultado en pantalla, pero con una estructura de código mucho más corta y fácil de leer.





6. **Interpretación:** La base de datos interpreta las letras asignadas (c, ca, a) como referencias directas a las tablas originales, evitando repetir nombres largos en cada línea.

7. **Aprendizaje:** Aprendí a optimizar la redacción de códigos largos para hacerlos más eficientes y evitar errores de sintaxis por nombres complejos.

8. **Aplicación en el mundo real:** En reportes corporativos extensos, utilizar abreviaturas estandarizadas para cruzar bases de datos complejas sin saturar el sistema.

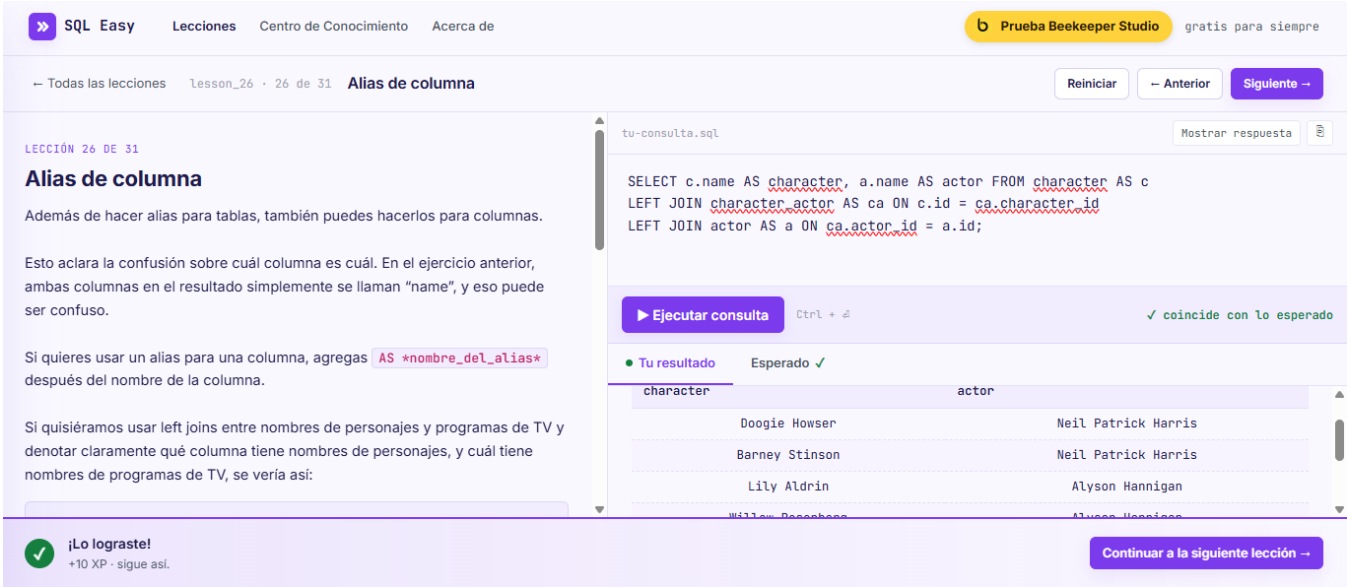
Ejercicio 26: Alias de columna

1. **Número y nombre del ejercicio:** Lección 26: Alias de columna
2. **Objetivo del ejercicio:** Renombrar los encabezados de las columnas en el resultado de la consulta para evitar confusiones cuando dos tablas comparten el mismo nombre de columna (como name).
3. **Concepto SQL:** Uso de AS directamente en las columnas seleccionadas (SELECT c.name AS character, a.name AS actor).
4. **Consulta ejecutada:** ¿Puedes usar left joins para emparejar nombres de personajes con los actores que los interpretan, y usar alias para llamar a las dos columnas devueltas *character* y *actor*? SELECT c.name AS character, a.name AS actor FROM character AS c

LEFT JOIN character_actor AS ca ON c.id = ca.character_id

LEFT JOIN actor AS a ON ca.actor_id = a.id;

5. **Resultado:** Una tabla limpia cuyos encabezados ya no dicen repetidamente "name", sino que muestran claramente **character** y **actor**.



6. **Interpretación:** El sistema toma los datos originales pero cambia la etiqueta visual de la columna para que el reporte final sea totalmente comprensible para cualquier lector.

7. **Aprendizaje:** Comprendí cómo pulir la presentación final de los datos para entregar reportes profesionales y limpios listos para su lectura ejecutiva.

8. **Aplicación en el mundo real:** En un software de administración, cambiar encabezados técnicos por nombres amigables como Nombre del Cliente y Fecha de Vencimiento en los reportes impresos para la gerencia.

Para el ejercicio 27 se observaron y usaron las siguientes tablas disponibles:



// TABLAS DISPONIBLES

rps · 3 | employees · 4

id PK int	name char	defeats_id int → rps.id
1	Rock	3
2	Paper	1
3	Scissors	2

// TABLAS DISPONIBLES 2 tab

rps · 3 | employees · 4

id PK int	name char	title char	boss_id int → employees.id
1	Patrick Smith	Software Engineer	2
2	Abigail Reed	Engineering Manager	3
3	Bob Carey	Director of Engineering	4
4	Maxine Tang	CEO	*NULL*

Ejercicio 27: Self joins (Unión de una tabla consigo misma)

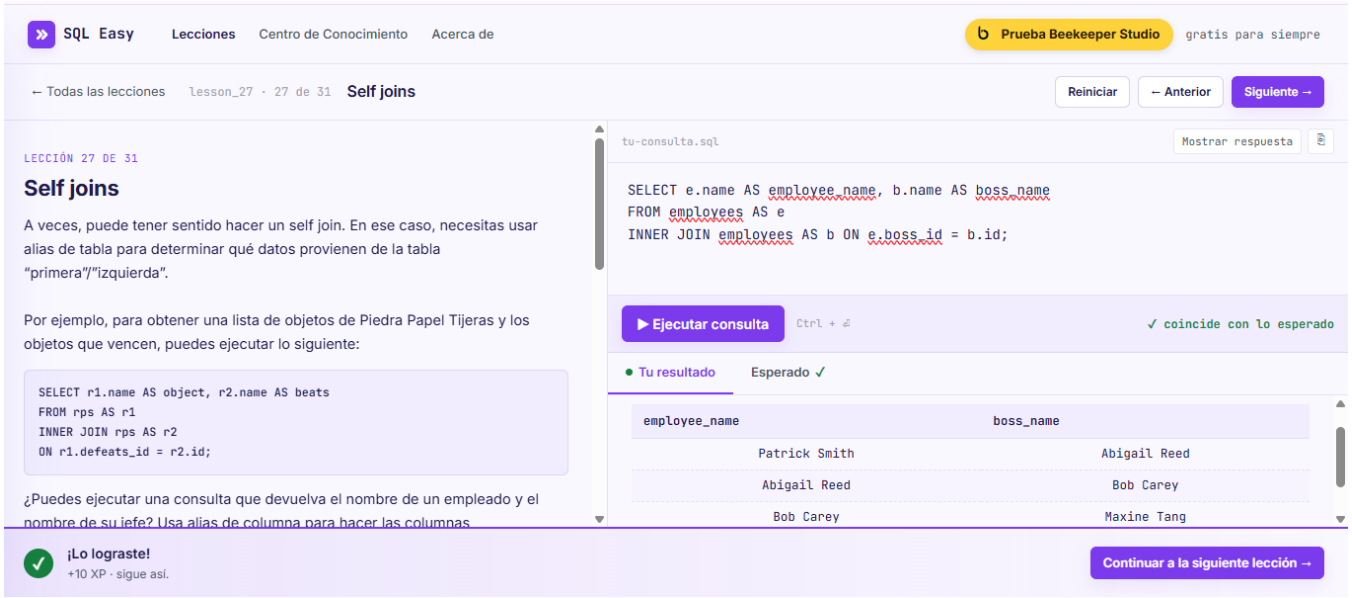
1. **Número y nombre del ejercicio:** Lección 27: Self joins
2. **Objetivo del ejercicio:** Mostrar el nombre de cada empleado junto con el nombre de su respectivo jefe, utilizando una tabla que se relaciona con ella misma.
3. **Concepto SQL:** Uso de un INNER JOIN donde una misma tabla se une consigo misma utilizando alias diferentes (por ejemplo, employees AS e y employees AS b).

4. **Consulta ejecutada:** ¿Puedes ejecutar una consulta que devuelva el nombre de un empleado y el nombre de su jefe? Usa alias de columna para hacer las columnas *employee_name* y *boss_name*.

SELECT e.name AS employee_name, b.name AS boss_name

FROM employees AS e INNER JOIN employees AS b ON e.boss_id = b.id;

5. **Resultado:** Una lista clara con dos columnas: el nombre del empleado y el nombre de su jefe correspondiente.



6. **Interpretación:** La base de datos trata a la tabla como si fueran dos distintas; busca el identificador del jefe (boss_id) dentro de la misma lista de IDs de los empleados para emparejarlos.

7. **Aprendizaje:** Comprendí que es posible cruzar datos internos de una sola tabla cuando existen jerarquías o relaciones de dependencia dentro del mismo grupo de registros.

8. **Aplicación en el mundo real:** En el organigrama de una empresa, estructurar los reportes jerárquicos para saber qué empleado le reporta directamente a qué supervisor o gerente.

Para el ejercicio 28 se observó y uso la siguiente tabla disponible:

// TABLAS DISPONIBLES

robots · 8

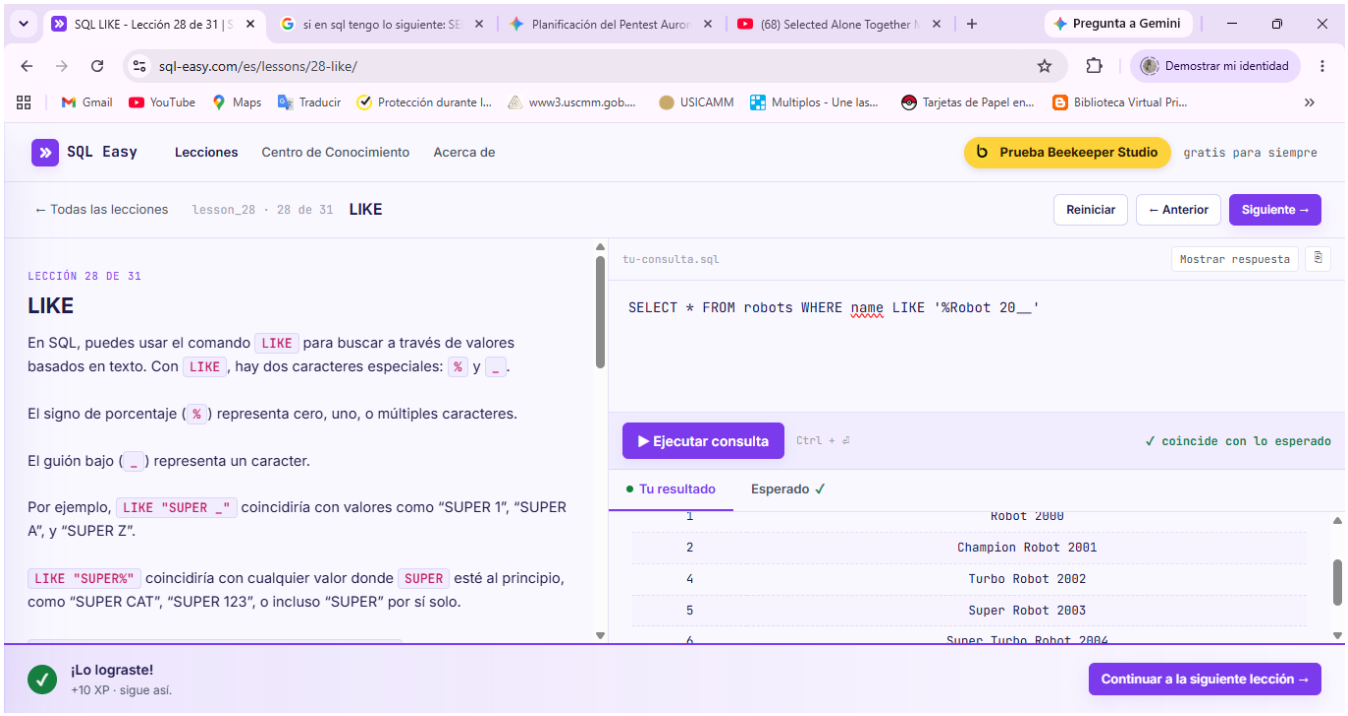
id PK int	name char
1	Robot 2000
2	Champion Robot 2001
3	Dragon
4	Turbo Robot 2002
5	Super Robot 2003
6	Super Turbo Robot 2004
7	Not A Robot
8	Unreleased Turbo Robot 2111

Ejercicio 28: LIKE (Búsqueda de patrones en texto)

1. **Número y nombre del ejercicio:** Lección 28: LIKE
2. **Objetivo del ejercicio:** Buscar registros de robots cuyos nombres sigan un patrón de texto específico utilizando comodines.
3. **Concepto SQL:** Uso del operador LIKE acompañado de los caracteres especiales % (que representa múltiples caracteres) y _ (que representa un carácter individual).

4. **Consulta ejecutada:** ¿Puedes ejecutar una consulta que devuelva “Robot” seguido de un año entre 2000 y 2099? (Así que 2015 es un valor válido al final, pero 2123 no lo es.) SELECT * FROM robots WHERE name LIKE '%Robot 20__';

5. **Resultado:** Los registros de los robots cuyos nombres coinciden exactamente con el patrón de búsqueda estructurado.



6. **Interpretación:** El sistema analiza las cadenas de texto buscando coincidencias parciales en lugar de requerir una palabra exacta, ignorando lo que esté antes y acotando los últimos espacios con los guiones bajos.

7. **Aprendizaje:** Aprendí a realizar búsquedas flexibles de texto cuando no recordamos el nombre exacto de un registro o cuando necesitamos filtrar por estructuras de códigos similares.

8. **Aplicación en el mundo real:** En un sistema de inventario, buscar productos o archivos escribiendo solo una parte del nombre o clave (por ejemplo, buscar todas las facturas que inicien con "FAC-2026").



Para el ejercicio 29 se observó y uso la siguiente tabla disponible:

// TABLAS DISPONIBLES 1 tabla

friends_of_pickles · 7

id PK int	name char	gender char	species char	height_cm int
1	Dave	male	human	180
2	Mary	female	human	160
3	Fry	male	cat	30
4	Leela	female	cat	25
5	Odie	male	dog	40
6	Jumpy	male	dog	35
7	Sneakers	male	dog	55

Ejercicio 29: CASE (Condicionales tipo IF / THEN)

1. **Número y nombre del ejercicio:** Lección 29: CASE
2. **Objetivo del ejercicio:** Crear una columna dinámica nueva llamada sound que asigne un sonido específico (talk, bark, meow) dependiendo de la especie de cada animal.
3. **Concepto SQL:** Uso de la estructura condicional CASE ... WHEN ... THEN ... ELSE ... END para transformar o evaluar datos sobre la marcha al momento de consultar.
4. **Consulta ejecutada:** ¿Puedes devolver los resultados con una columna llamada *sound* que devuelva “talk” para humanos, “bark” para perros, y “meow” para gatos?

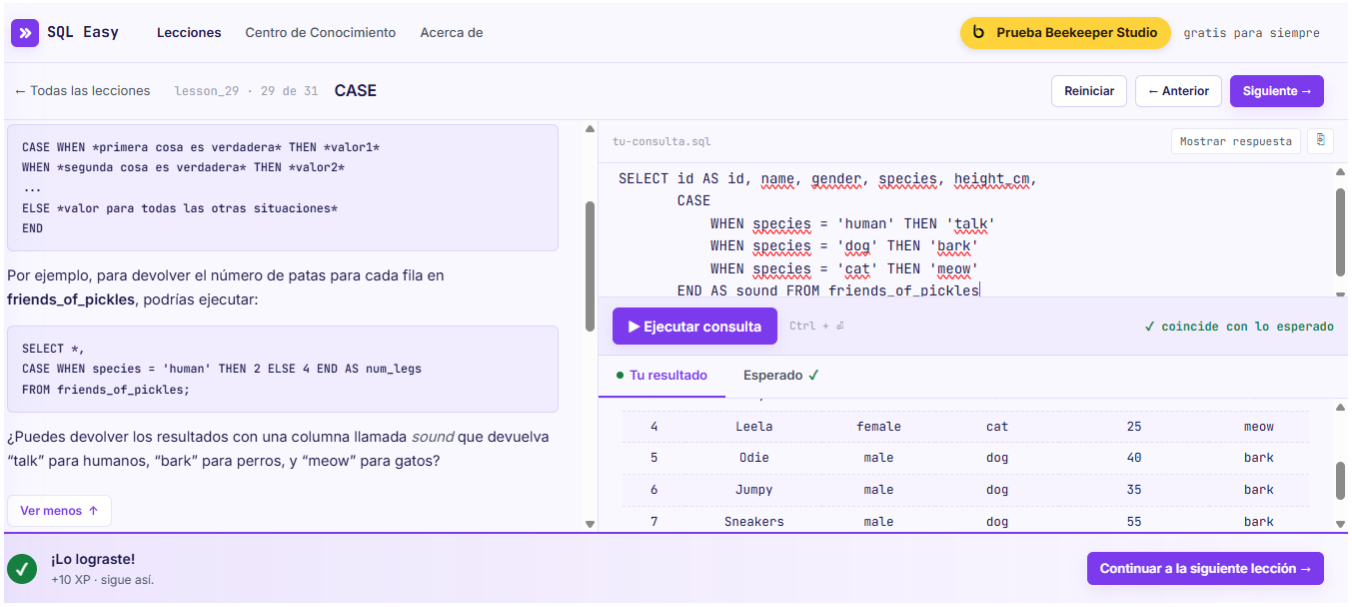
```
SELECT id AS id, name, gender, species, height_cm, CASE
    WHEN species = 'human' THEN 'talk'
    WHEN species = 'dog' THEN 'bark'
```

```
SELECT *
FROM datos
WHERE
condicion = 'valor';
```

id	nombre	rol	estado
1	Ana	Analista	Activo
2	Luis	Pentester	Activo
3	Sofía	DevSecOps	Activo

```
    WHEN species = 'cat' THEN 'meow'
END AS sound
FROM friends_of_pickles;
```

5. **Resultado:** Una tabla que incluye una columna adicional con el comportamiento o sonido traducido para cada especie.



6. **Interpretación:** La base de datos evalúa fila por fila la especie y aplica una regla condicional para arrojar un resultado personalizado en tiempo real sin modificar la tabla original.

7. **Aprendizaje:** Comprendí cómo aplicar lógica de programación condicional dentro de una consulta SQL con clasificaciones automáticas.

8. **Aplicación en el mundo real:** En un sistema financiero, clasificar automáticamente a los clientes como "Cliente VIP" o "Cliente Regular" según el volumen de sus compras anuales al generar un reporte.



Para el ejercicio 30 se observó y uso la siguiente tabla disponible:

// TABLAS DISPONIBLES 1 tal

robots · 8

id PK int	name char	location char
1	R2000 - Robot 2000	New City, NY
2	R2001 - Champion Robot 2001	Palo Alto, CA
3	D0001 - Dragon	New York City, NY
4	R2002 - Turbo Robot 2002	Spring Valley, NY
5	R2003 - Super Robot 2003	Nyack, NY
6	R2004 - Super Turbo Robot 2004	Tampa, FL
7	N0001 - Not A Robot	Seattle, WA
8	U2111 - Unreleased Turbo Robot 2111	Buffalo, NY

Ejercicio 30: SUBSTR (Extraer subcadenas de texto)

1. **Número y nombre del ejercicio:** Lección 30: SUBSTR
2. **Objetivo del ejercicio:** Filtrar robots cuya ubicación termine específicamente en el estado de Nueva York (NY) extrayendo una parte de la cadena de texto.
3. **Concepto SQL:** Uso de la función SUBSTR(columna, inicio, caracteres) para recortar y analizar fragmentos específicos de un texto, combinada con índices negativos para contar desde el final.
4. **Consulta ejecutada:** ¿Puedes devolver todos los robots que están ubicados en NY? SELECT id, name, location FROM robots

WHERE SUBSTR(location, -2) = 'NY';

5.

6. **Resultado:** Los registros de los robots localizados en ciudades cuyo código de estado final sea "NY".



7. **Interpretación:** La función lee el campo de ubicación, toma los últimos dos caracteres contando desde el final (gracias al índice negativo -2) y los compara con el filtro deseado.

8. **Aprendizaje:** Aprendí a manipular datos de texto cuando la información viene combinada en una sola celda

9. **Aplicación en el mundo real:** En una base de datos de direcciones postales, extraer únicamente los códigos postales o abreviaturas de estados para hacer envíos regionales o segmentaciones geográficas.



Para el ejercicio 31 se observó y uso la siguiente tabla disponible:

// TABLAS DISPONIBLES — 1 tabla

fighters · 4

id PK int	name char	gun char	sword char	tank char
1	US Marine	Colt 9mm SMG	Swiss Army Knife	M1A1
2	John Wilkes Booth	.44 caliber Derringer	*NULL*	*NULL*
3	Zorro	*NULL*	Sword of Zorro	*NULL*
4	Innocent Bystander	*NULL*	*NULL*	*NULL*

Ejercicio 31: COALESCE (Devolver el primer valor no nulo)

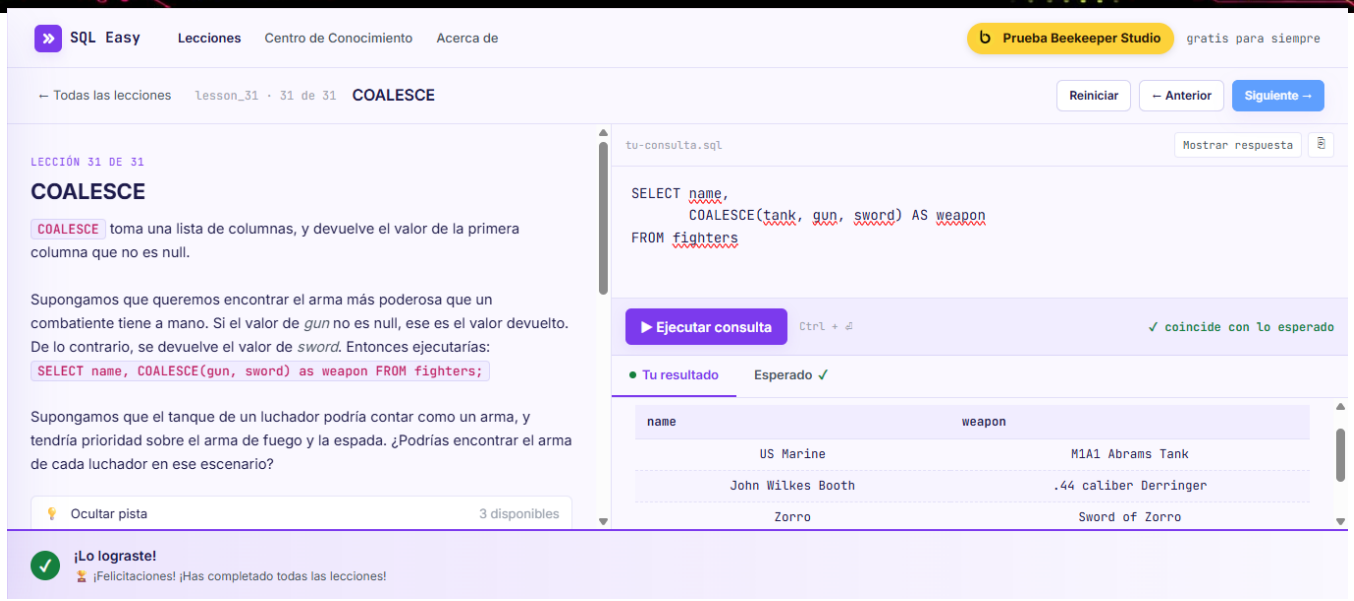
1. **Número y nombre del ejercicio:** Lección 31: COALESCE
2. **Objetivo del ejercicio:** Encontrar y mostrar el arma principal disponible de cada combatiente, evaluando un orden de prioridad entre varias opciones de columnas.
3. **Concepto SQL:** Uso de la función COALESCE(col1, col2, col3) la cual revisa una lista de columnas en orden y devuelve **el primer valor que no esté vacío (NULL)**.
4. **Consulta ejecutada:** Supongamos que el tanque de un luchador podría contar como un arma, y tendría prioridad sobre el arma de fuego y la espada. ¿Podrías encontrar el arma de cada luchador en ese escenario?

SELECT name, COALESCE(tank, gun, sword) AS weapon

FROM fighters;

5. **Resultado:** Una lista que asigna correctamente el arma principal de cada combatiente (por ejemplo, el tanque si lo tiene, de lo contrario la pistola o la espada).





6. **Interpretación:** El sistema revisa la primera columna de armas; si está vacía (NULL), pasa a la segunda, y si tampoco tiene datos, toma la tercera, garantizando que siempre se devuelva una opción válida.

7. **Aprendizaje:** Comprendí cómo gestionar datos faltantes ofreciendo alternativas jerárquicas automáticas para que los reportes nunca arrojen espacios vacíos

8. **Aplicación en el mundo real:** En un sistema de contactos de clientes, mostrar el primer teléfono disponible evaluando en orden: "Teléfono móvil", "Teléfono de oficina" o "Teléfono de casa".

CONCLUSIÓN



Este recorrido de 31 ejercicios en *SQL Easy* ha representado un avance en mi formación tecnológica. Iniciar con consultas básicas me permitió comprender la esencial de los datos, mientras que enfrentar los temas avanzados especialmente a partir del ejercicio 21 con las uniones (JOIN), subconsultas y transformaciones de texto exigió un nivel mayor de análisis y estructuración.

Esta práctica me demostró que saber manejar bases de datos no es solo una habilidad técnica, sino una necesidad estratégica. En cualquier organización o área ya sea corporativa, comercial, tecnológica, administrativa o de ciberseguridad, la información que se maneja en datos es el activo más valioso que posee una empresa. Entender cómo estructurarla, consultarla y analizarla nos permite utilizarla a nuestro favor para la toma de decisiones, pero sobre todo para cuidarlos y protegerlos, ya que son lo más valioso que tiene una empresa. Este reto me enseño bases sólidas para seguir creciendo en el ámbito profesional.